Università degli Studi di Milano

Corso di Laurea Magistrale in Sicurezza Informatica

Relazione del corso di

Sicurezza nelle architetture Data-Intensive

GraphMate

Graph Neural Networks per la predizione
e il reranking di mosse scacchistiche

Studenti

Alessandro Zappavigna

Mattia Ippoliti

Samuele Alfrondi

Anno Accademico 2026/2027

Indice

1 Introduzione **1**
1.1 Contesto e motivazione 1
1.2 Obiettivi e domande di ricerca 1
1.3 Organizzazione della relazione 2

2 Background e lavori correlati **3**
2.1 Posizioni, mosse e problemi di matto 3
2.2 Reti neurali su grafi e attenzione 3
2.3 Informazione temporale e rapporto con TimeGNN 4
2.4 Modelli linguistici come baseline 4

3 Dataset e preparazione dei dati **5**
3.1 Sorgenti dei dati 5
3.1.1 Puzzle Lichess 5
3.1.2 Partite ausiliarie e benchmark esterno 5
3.2 Pulizia e costruzione del task supervisionato 6
3.3 Bilanciamento e suddivisione 6
3.4 Codifica della mossa e generazione dei grafi 7
3.5 Variante con timing sintetico 7

4 Rappresentazione a grafo **8**
4.1 Dalla scacchiera al grafo 8
4.2 Rappresentazione dei nodi 9
4.3 Rappresentazione degli archi 10
4.4 Feature globali del grafo 12
4.5 PyTorch Geometric Data Object 12
4.6 Generazione e persistenza del dataset a grafo 13

5 Architetture dei modelli **15**
5.1 Panoramica della famiglia di modelli 15
5.2 Encoder Graph Attention condiviso 16
5.3 Modelli a vocabolario fisso: A e A2 17

5.3.1 Modello A: classificatore sul vocabolario globale 17
5.3.2 Modello A2: classificatore con mascheramento delle mosse illegali 18
5.4 Scoring delle mosse legali: Modello A3 18
5.5 Modello B e informazione temporale 19
5.6 Reranking delle posizioni post-mossa: Modello A4 20
5.7 Confronto architetturale 21

6 Metodologia sperimentale **22**
6.1 Disegno sperimentale 22
6.2 Addestramento e selezione dei checkpoint 22
6.3 Protocollo di valutazione Lichess 23
6.4 Protocollo dei modelli linguistici 23
6.5 Benchmark esterno 23
6.6 Metriche e analisi statistica 24
6.7 Limiti del protocollo 24

7 Risultati **25**
7.1 Panoramica delle valutazioni 25
7.2 Risultati sul benchmark Lichess 25
7.3 Contributo delle informazioni temporali 26
7.4 Prestazioni in funzione della profondità del matto 27
7.5 Analisi per fascia di rating 28
7.6 Confronto con le baseline LLM su Lichess 29
7.7 Risultati sul benchmark esterno YACPDB 30
7.8 Generalizzazione tra Lichess e YACPDB 31
7.9 Sintesi delle misure 31

8 Discussione **32**
8.1 Vincoli scacchistici e spazio decisionale 32
8.2 Reranking post-mossa e limite di recupero 32
8.3 Timing e confronto con i modelli linguistici 33
8.4 Generalizzazione e limiti sperimentali 33

9 Conclusioni **35**

Bibliografia **36**

Appendici **37**

A Configurazioni ufficiali, iperparametri e protocollo di addestramento 38
A.1 Politica di addestramento comune 38
A.2 Sintesi delle architetture dei modelli 39
A.3 Inventario dei checkpoint congelati 41
A.4 Note per modello 41

B Rappresentazione a grafo e schema delle feature 43
B.1 Indicizzazione delle case e layout dei tensori 43
B.2 Feature dei nodi 44
B.3 Feature degli archi 44
B.4 Feature globali 45
B.5 Rappresentazioni delle mosse candidate 46

C Pipeline dei dati, suddivisioni e timing sintetico 47
C.1 Semantica dei puzzle Lichess 47
C.2 Vocabolario delle mosse e conteggi dei grafi 48
C.3 Dataset di timing sintetico 48

D Costruzione e verifica del benchmark di composizioni classiche 50
D.1 Acquisizione ed eleggibilità 50
D.2 Selezione, fingerprint e ciclo di congelamento 51
D.3 Verifica Popeye e chiavi accettate 52
D.4 Risultati della valutazione sulle composizioni classiche 52

E Protocollo di valutazione dei modelli linguistici 54
E.1 Identità del runtime e prompt 54
E.2 Parser rigoroso e tassonomia degli esiti 55
E.3 Parser rilassato diagnostico 55
E.4 Valutazione sulle composizioni classiche e reference-line 56
E.5 Validità delle esecuzioni ed esclusioni 56

F Riproducibilità e inventario degli artefatti 57
F.1 Classi di artefatti 57
F.2 Tre casi d'uso per la riproduzione 58
F.3 Interfaccia ufficiale di addestramento 59
F.4 Riproduzione di checkpoint e valutazioni 59
F.5 Ambiente e limiti di un clone fresco 59

Sommario

Questo lavoro presenta *GraphMate*, un sistema per la predizione di mosse tattiche negli scacchi basato su *Graph Neural Networks* (GNN). L'obiettivo del progetto è studiare l'efficacia di una rappresentazione strutturata della scacchiera, nella quale una posizione non viene trattata come una semplice matrice di celle, ma come un grafo capace di descrivere esplicitamente le relazioni scacchistiche tra le case e i pezzi.

Il dataset principale è costruito a partire dai puzzle pubblici di Lichess, selezionando problemi di matto forzato da *mate-in-1* a *mate-in-5*. Dopo le operazioni di filtraggio, validazione, deduplicazione e bilanciamento, ciascun problema viene formulato come un task supervisionato di predizione della prima mossa risolutiva. Ogni posizione viene quindi trasformata in un grafo diretto composto da 64 nodi, corrispondenti alle case della scacchiera, e da archi che rappresentano relazioni quali mosse legali, attacchi, difese, inchiodature e scacchi. Ai nodi e agli archi sono associate feature locali, integrate da feature globali che descrivono lo stato complessivo della posizione.

Su questa rappresentazione viene sviluppata e confrontata una famiglia di modelli basati su un encoder *Graph Attention Network*. La baseline formula il problema come classificazione su un vocabolario globale di mosse UCI; successive varianti introducono il mascheramento delle mosse illegali e, soprattutto, uno scoring diretto delle sole mosse legali disponibili nella posizione. Vengono inoltre analizzate due estensioni: l'integrazione di informazioni temporali sintetiche e un'architettura di *reranking* a due stadi, nella quale le migliori candidate proposte dal modello vengono rivalutate considerando esplicitamente la posizione risultante dopo la loro applicazione.

La valutazione comprende sia dati derivati dalla distribuzione Lichess sia un benchmark esterno di composizioni scacchistiche classiche, utilizzato per analizzare la capacità di generalizzazione dei modelli. Su quest'ultimo benchmark, il modello con reranking post-mossa raggiunge un'accuratezza Top-1 del 15,5%, rispetto all'11,0% del modello candidate-based di base, mostrando il beneficio dell'analisi esplicita dello stato risultante dalla mossa. Come ulteriore termine di confronto vengono valutati modelli linguistici locali mediante un protocollo rigoroso di generazione di una singola mossa UCI; in tale configurazione le prestazioni risultano nettamente inferiori a quelle dei modelli specializzati.

I risultati evidenziano come l'integrazione della struttura del dominio nello spazio decisionale, attraverso la rappresentazione a grafo, il vincolo esplicito di legalità e l'analisi delle posizioni successive alla mossa, costituisca un approccio promettente per la modellazione neurale di problemi tattici negli scacchi. Il progetto fornisce

inoltre una pipeline riproducibile che comprende preprocessing dei dati, costruzione dei grafi, addestramento, gestione dei checkpoint e protocolli di valutazione.

Capitolo 1

Introduzione

1.1 Contesto e motivazione

La scelta di una mossa scacchistica dipende da relazioni che non sono puramente geometriche: mobilità, attacchi, difese, inchiodature e vincoli di legalità collegano case anche lontane sulla scacchiera. GraphMate studia se una rappresentazione esplicita di queste relazioni, elaborata mediante reti neurali a grafo (*Graph Neural Networks*, GNN), possa sostenere la predizione della prima mossa risolutiva di un problema tattico di matto.

I puzzle scacchistici costituiscono un caso di studio utile perché associano a una posizione una soluzione verificabile e impongono regole formali sullo spazio delle azioni. Il progetto confronta progressivamente una classificazione su un vocabolario globale di mosse, l'applicazione di vincoli di legalità, lo scoring diretto delle mosse legali e il reranking basato sulla posizione successiva alla mossa. In parallelo, valuta una variante che integra un segnale temporale sintetico, con l'obiettivo di verificare se la specifica informazione di timing implementata migliori la scelta della mossa.

Come termine di confronto esterno alla famiglia GNN, GraphMate include due modelli linguistici locali sottoposti a un protocollo controllato di generazione di una singola mossa UCI. Il confronto non intende misurare ogni possibile capacità scacchistica dei modelli linguistici: valuta sistemi completi nel compito e nel formato fissati dal protocollo. La generalizzazione dei modelli viene inoltre esaminata su composizioni classiche YACPDB, distinte dalla distribuzione dei puzzle Lichess usata per lo sviluppo principale.

1.2 Obiettivi e domande di ricerca

L'obiettivo generale è costruire e valutare una pipeline riproducibile che trasformi posizioni scacchistiche in grafi, apprenda a ordinare mosse candidate e renda trac-

ciabile il passaggio dai dati ai risultati congelati. Gli esperimenti rispondono a due domande di ricerca:

RQ1. Per quali profondità di matto il modello GNN con timing fornisce indicazioni di mossa migliori delle baseline linguistiche selezionate?

RQ2. Nell'esperimento controllato, l'integrazione del timing sintetico migliora la prestazione rispetto al modello GNN equivalente senza timing?

La prima domanda confronta il Modello B con Qwen 3.5 4B e 9B sul benchmark Lichess, l'unico per il quale sono disponibili gli ingressi temporali previsti. La seconda isola invece il contributo del timing attraverso il confronto appaiato tra il Modello B e A3. Il ruolo dei modelli A, A1, A2 e A4 è ampliare l'analisi delle scelte architetturali, in particolare dei vincoli di legalità e della valutazione post-mossa.

1.3 Organizzazione della relazione

Il Capitolo 2 introduce le nozioni scacchistiche e neurali necessarie. I Capitoli 3 e 4 descrivono rispettivamente i dati e la rappresentazione a grafo; il Capitolo 5 presenta la famiglia di modelli. Il Capitolo 6 definisce il disegno sperimentale, mentre il Capitolo 7 riporta le misure. I Capitoli 8 e 9 ne discutono il significato e formulano le conclusioni. Le appendici raccolgono specifiche implementative, protocolli, iperparametri e informazioni di riproducibilità.

Capitolo 2

Background e lavori correlati

2.1 Posizioni, mosse e problemi di matto

Una posizione scacchistica è descritta nel progetto mediante la notazione FEN (*Forsyth–Edwards Notation*), che codifica disposizione dei pezzi, lato al tratto, diritti di arrocco, eventuale casa en passant e contatori di mossa. Le mosse sono espresse in formato UCI: la stringa identifica casa di origine, casa di destinazione ed eventuale promozione, per esempio `e2e4` o `e7e8q`. Il formato è compatto e può essere verificato automaticamente rispetto alle regole della posizione.

Un problema *mate-in-N* richiede una prima mossa che garantisca il matto entro il numero dichiarato di mosse del solutore, contro ogni difesa ammessa. Nel dataset Lichess la FEN precede una mossa di preparazione dell'avversario: la posizione mostrata al solutore si ottiene applicando la prima mossa della sequenza, mentre la seconda è la prima mossa della soluzione [1]. GraphMate apprende questa singola mossa target; non genera autonomamente l'intera variante fino al matto.

2.2 Reti neurali su grafi e attenzione

Una rete neurale a grafo aggiorna la rappresentazione di ciascun nodo aggregando informazioni provenienti dai nodi collegati. Ripetendo l'operazione su più strati, ogni nodo incorpora un contesto strutturale progressivamente più ampio. Il *pooling* riassume poi le rappresentazioni locali in un vettore del grafo, utilizzabile per una decisione relativa all'intera posizione. Questa impostazione generalizza l'idea delle convoluzioni su grafi introdotta, tra gli altri, da Kipf e Welling [2].

Le *Graph Attention Networks* (GAT) assegnano pesi appresi ai vicini invece di combinarli tutti nello stesso modo [3]. Intuitivamente, quando il modello aggiorna la rappresentazione di una casa, può attribuire maggiore rilevanza alle relazioni più informative per quella posizione. L'attenzione è calcolata separatamente da più

teste e i risultati vengono combinati; ciò consente di apprendere prospettive differenti sulle stesse relazioni. In GraphMate gli archi portano inoltre attributi scacchistici, cosicche il messaggio dipende sia dai nodi coinvolti sia dal tipo di relazione. Il Capitolo 4 specifica le feature effettive e il Capitolo 5 descrive l'encoder adottato.

2.3 Informazione temporale e rapporto con TimeGNN

Le reti temporali su grafi introducono variabili di tempo nella costruzione o nell'elaborazione delle relazioni. TimeGNN, per esempio, studia rappresentazioni temporali dinamiche per serie multivariate [4]. La libreria TimeGNN presente nel progetto ha fornito riferimenti architetturali e tecniche di implementazione basate su PyTorch Geometric, come layer di attenzione, gestione degli attributi degli archi e organizzazione dell'addestramento.

GraphMate non esegue però il framework TimeGNN completo e non tratta la scacchiera come un event log dinamico. I suoi grafi rappresentano singole posizioni; nel Modello B il timing sintetico è codificato da un piccolo encoder e concatenato al contesto globale usato per valutare le mosse candidate. Si tratta quindi di un adattamento di idee e tecniche, progettato per un compito graph-level differente.

2.4 Modelli linguistici come baseline

Un modello linguistico può ricevere la FEN in un prompt e generare testo contenente una mossa. Diversamente dai modelli GraphMate, non riceve in ingresso il grafo scacchistico costruito dal progetto e non applica internamente il generatore di mosse legali della pipeline GNN. Per ottenere un confronto verificabile, il protocollo congela prompt, parametri di generazione e parser e richiede una singola mossa UCI. La valutazione separa output non interpretabili, mosse illegali, mosse legali errate e risposte corrette; i dettagli sono riportati nel Capitolo 6 e nell'Appendice E.

Capitolo 3

Dataset e preparazione dei dati

3.1 Sorgenti dei dati

3.1.1 Puzzle Lichess

La sorgente principale è il database pubblico dei puzzle Lichess, distribuito come CSV compresso con Zstandard. Ogni record contiene, tra gli altri campi, l'identificativo del puzzle, una posizione FEN, una sequenza di mosse UCI, il rating e i temi assegnati. Lichess documenta inoltre una semantica importante: la FEN precede la mossa dell'avversario che avvia il puzzle, mentre la soluzione del giocatore comincia dalla seconda mossa della sequenza [1].

GraphMate seleziona i temi `mateIn1`, `mateIn2`, `mateIn3`, `mateIn4` e `mateIn5`. La pipeline ufficiale registra 1.937.001 righe dopo il filtraggio tematico e 1.931.434 dopo la pulizia. Questi valori provengono dai file e dal log della pipeline finale; sostituiscono i conteggi di una precedente esecuzione esplorativa, non usata per gli split congelati.

3.1.2 Partite ausiliarie e benchmark esterno

Il progetto include anche un campione di partite Lichess in formato PGN, usato nella fase esplorativa per studiare i clock e i metadati di gioco. Questi dati non sono allineati uno-a-uno ai puzzle e non forniscono i tempi usati dal Modello B: gli esperimenti finali impiegano timing sintetico associato ai puzzle, come descritto nella Sezione 3.5.

La valutazione esterna usa invece `yacpdb_classic_v1`, un insieme congelato di 200 composizioni di matto diretto provenienti da YACPDB, con 20 problemi per profondità da #1 a #10. Il benchmark è escluso da addestramento, selezione degli iperparametri e scelta dei checkpoint. La sua costruzione, la verifica indipendente con Popeye e la politica delle chiavi accettate sono documentate nell'Appendice D.

3.2 Pulizia e costruzione del task supervisionato

La pulizia valida le FEN, richiede rating nell'intervallo 400–3500, profondità supportata e sequenza non vuota, quindi rimuove i duplicati individuati dalla coppia FEN–sequenza di mosse. Per costruire l'esempio supervisionato, la pipeline:

1. conserva la FEN grezza come `OriginalFEN`;
2. applica `Moves[0]` come mossa di preparazione legale;
3. salva la posizione risultante come FEN del solutore;
4. usa `Moves[1]` come `TargetMove`.

Sono scartati i record con meno di due mosse, FEN non valida, mossa di preparazione malformata o illegale e target malformato o illegale nella posizione del solutore. Il compito consiste dunque nel predire la prima mossa risolutiva a partire dalla posizione effettivamente presentata al giocatore, non nel generare l'intera linea di matto.

3.3 Bilanciamento e suddivisione

Per limitare lo sbilanciamento della sorgente, vengono campionati al massimo 20.000 esempi per profondità. Le classi mate-in-1–mate-in-4 raggiungono tale soglia, mentre per mate-in-5 sono disponibili 6.198 esempi. Il dataset bilanciato contiene quindi 86.198 record.

La suddivisione 80/10/10 usa random state 42 ed è stratificata per profondità. La Tabella 3.1 distingue le righe CSV dai grafi effettivamente generati.

Tabella 3.1: Split finali e campioni utilizzabili dai modelli a grafo.

Split	Righe CSV	Grafi PyG	Target OOV esclusi
Addestramento	68.958	68.958	0
Validazione	8.620	8.612	8
Test	8.620	8.610	10

I primi quattro livelli rappresentano ciascuno circa il 23,20% di ogni split; mate-in-5 rappresenta circa il 7,19%. I rating medi sono rispettivamente 1.338,84, 1.337,61 e 1.338,05 per addestramento, validazione e test. La vicinanza di queste statistiche è coerente con la suddivisione stratificata, ma non elimina altre possibili differenze tra esempi.

3.4 Codifica della mossa e generazione dei grafi

Il target resta una stringa UCI fino alla codifica numerica. Il vocabolario è costruito esclusivamente sull'insieme di addestramento e contiene 1.786 mosse distinte. Questa scelta evita di usare validazione o test per definire le classi, ma produce 8 target fuori vocabolario in validazione e 10 nel test. Tali record rimangono nei CSV e vengono esclusi dalla generazione PyG; per questo le valutazioni congelate usano 8.610 grafi di test.

Ogni record utilizzabile viene trasformato nel grafo descritto nel Capitolo 4. I grafi sono salvati in shard da 1.000 elementi e accompagnati da un manifesto che registra schema, conteggi, file e anomalie. I conteggi dettagliati e il funzionamento del loader sharded sono riportati nell'Appendice C.

3.5 Variante con timing sintetico

Il Modello B usa gli stessi esempi e la stessa appartenenza agli split, arricchiti con il tempo sintetico della mossa precedente, il tempo sintetico della decisione corrente e un indicatore che ne dichiara la natura sintetica. La generazione è deterministica rispetto a seed 42, identificativo del puzzle e stream casuale; la distribuzione è condizionata dal rating e i secondi sono limitati all'intervallo $[0{,}5, 120]$.

La normalizzazione applica uno z-score a $\log(1+t)$, con media e deviazione standard stimate sul solo insieme di addestramento. Nessun esempio finale contiene un tempo umano reale. Questa distinzione è essenziale: l'esperimento misura l'effetto della specifica codifica sintetica implementata, non l'utilità generale del tempo di riflessione. Equazioni, parametri e campi persistiti sono riportati nell'Appendice C.

Capitolo 4

Rappresentazione a grafo

4.1 Dalla scacchiera al grafo

Il Capitolo 3 ha descritto la costruzione del dataset supervisionato fino alla disponibilità, per ogni esempio, di una posizione in formato FEN, di una mossa target in formato UCI e dei metadati associati al puzzle. Il passo successivo consiste nel trasformare tale posizione in una struttura dati adatta a modelli basati su grafi. In questa rappresentazione, una posizione scacchistica viene modellata come un grafo diretto

$$G = (V, E),$$

dove l'insieme dei nodi V corrisponde alle case della scacchiera e l'insieme degli archi E codifica relazioni scacchistiche calcolate sulla posizione.

La scelta fondamentale della rappresentazione è associare un nodo a ciascuna delle 64 case della scacchiera. Di conseguenza, il numero di nodi è fisso e non dipende dal numero di pezzi presenti nella posizione. Anche le case vuote sono quindi rappresentate esplicitamente: esse non contengono un pezzo, ma mantengono coordinate, informazioni sugli attacchi ricevuti e possono essere destinazioni di mosse legali. Questo aspetto è importante perché, negli scacchi, molte relazioni rilevanti non riguardano soltanto le case occupate.

Gli archi non rappresentano semplicemente l'adiacenza geometrica tra case vicine. Al contrario, essi descrivono relazioni scacchistiche dirette, come mosse legali, attacchi, difese, inchiodature e scacchi al re. La direzione dell'arco ha quindi un significato: per esempio, una mossa legale viene rappresentata dalla casa di partenza alla casa di arrivo, mentre una relazione di attacco va dal pezzo attaccante al pezzo avversario attaccato. Poiché più relazioni possono valere per la stessa coppia ordinata di case, gli archi sono rappresentati tramite vettori multi-label: invece di creare più archi duplicati, la stessa coppia di nodi può avere più componenti attive nel proprio vettore di attributi.

Il flusso concettuale della trasformazione è riassunto nella Figura 4.1. La FEN viene interpretata come posizione scacchistica, da cui vengono estratte feature dei nodi, feature degli archi e feature globali. In parallelo, la mossa target viene codificata come label numerica. Il risultato finale è un oggetto compatibile con PyTorch Geometric, che aggrega struttura del grafo, feature, label e metadati.

```
FEN + TargetMove → chess.Board → Node / Edge / Global Features
                 → Target Encoding → PyG Data
```

Figura 4.1: Pipeline concettuale dalla posizione FEN alla rappresentazione a grafo.

4.2 Rappresentazione dei nodi

Ogni nodo rappresenta una casa della scacchiera ed è descritto da un vettore di 15 feature numeriche. La rappresentazione combina informazione statica, come tipo di pezzo e coordinate, con informazione tattica calcolata sulla posizione, come attacchi, mobilità legale e stato di inchiodatura. La Tabella 4.1 riporta il layout completo.

Indice	**Feature**	**Significato**	**Codifica / normalizzazione**
0	pawn	pezzo pedone	one-hot, 1.0 se pedone
1	knight	pezzo cavallo	one-hot, 1.0 se cavallo
2	bishop	pezzo alfiere	one-hot, 1.0 se alfiere
3	rook	pezzo torre	one-hot, 1.0 se torre
4	queen	pezzo donna	one-hot, 1.0 se donna
5	king	pezzo re	one-hot, 1.0 se re
6	color	colore del pezzo	1.0 bianco, 0.0 nero o vuoto
7	occupied	casa occupata	1.0 se occupata, 0.0 se vuota
8	row	rank della casa	rank/7
9	column	file della casa	file/7
10	attacked_by_white	attaccanti bianchi	conteggio grezzo
11	attacked_by_black	attaccanti neri	conteggio grezzo
12	legal_mobility	mosse legali dalla casa	conteggio grezzo
13	is_pinned	pezzo inchiodato	1.0 se pinned, altrimenti 0.0
14	piece_value	valore materiale	pedone 1, cavallo/alfiere 3, torre 5, donna 9, re 0

Tabella 4.1: Layout delle feature associate a ciascun nodo della scacchiera.

Le prime sei componenti codificano il tipo di pezzo tramite one-hot encoding. Una casa vuota ha quindi zeri in tutte queste posizioni. La feature di colore usa il

valore 1.0 per un pezzo bianco e 0.0 per un pezzo nero; anche una casa vuota ha valore 0.0. Per distinguere una casa vuota da una casa occupata da un pezzo nero, la rappresentazione include esplicitamente la feature `occupied`. Una casa vuota ha dunque one-hot nullo, colore 0.0, occupazione 0.0 e valore materiale 0.0, ma conserva comunque coordinate e informazioni sugli attacchi ricevuti.

Le coordinate della casa sono rappresentate tramite rank e file normalizzati nell'intervallo $[0, 1]$, dividendo il valore discreto per 7. Questa normalizzazione mantiene l'informazione geometrica della casa senza introdurre una codifica categorica separata per riga e colonna.

Le feature `attacked_by_white` e `attacked_by_black` contano il numero di pezzi bianchi o neri che attaccano la casa. Si tratta di conteggi grezzi, non normalizzati, e vengono calcolati per tutte le case, indipendentemente dal fatto che siano occupate. In questo modo una casa vuota ma tatticamente rilevante, per esempio una casa di destinazione sotto controllo, può comunque ricevere informazione strutturale.

La feature `legal_mobility` richiede una precisazione. Non indica semplicemente la mobilità del pezzo del lato al tratto nella posizione corrente. La procedura calcola, per ogni colore, le mosse legali ottenute impostando temporaneamente quel colore come lato al tratto su una copia della posizione. Per ciascuna mossa legale incrementa il conteggio della casa di partenza. La mobilità finale di una casa è quindi la somma dei conteggi ottenuti considerando entrambi i colori. Questa scelta produce una misura locale della potenziale attività della casa, separata dal solo turno corrente.

Infine, `is_pinned` indica se il pezzo presente sulla casa è inchiodato secondo la logica scacchistica della posizione. Per le case vuote il valore è sempre 0.0. Il valore materiale usa la scala classica pedone 1, cavallo e alfiere 3, torre 5, donna 9 e re 0; anche le case vuote hanno valore 0.

Il tensore delle feature dei nodi ha quindi forma $[64, 15]$: 64 nodi, uno per casa, e 15 feature per nodo.

4.3 Rappresentazione degli archi

Gli archi del grafo sono diretti e vengono creati quando tra due case della scacchiera esiste almeno una relazione scacchistica fra quelle implementate. Ogni arco possiede un vettore di attributi di dimensione 5, riportato nella Tabella 4.2. Le componenti del vettore sono multi-label: più valori possono essere attivi simultaneamente per lo stesso arco.

La relazione `legal_move` viene creata per ogni mossa legale del lato al tratto nella posizione corrente. L'arco è diretto dalla casa di partenza alla casa di arrivo. Le relazioni `attack` e `defend` derivano invece dagli attacchi dei pezzi: se un pezzo

Indice	Relazione	Significato	Direzione dell'arco
0	legal_move	mossa legale del lato al tratto	partenza → arrivo
1	attack	attacco a un pezzo avversario	attaccante → pezzo avversario
2	defend	difesa di un pezzo alleato	difendente → pezzo alleato
3	pin	relazione di inchiodatura	slider inchiodante → pezzo inchiodato
4	check_line	scacco al re del lato al tratto	checker → re

Tabella 4.2: Layout delle feature associate agli archi diretti del grafo.

attacca una casa occupata da un pezzo avversario, viene creato un arco di attacco; se la casa attaccata contiene un pezzo dello stesso colore, viene creato un arco di difesa. Le case vuote attaccate non generano archi di questo tipo, anche se possono essere coinvolte in archi di mossa legale.

La semantica della relazione `pin` è implementativa e va intesa con precisione. Il codice esplora le otto direzioni principali a partire da ciascun re. Se lungo una direzione incontra prima un pezzo dello stesso colore del re e successivamente un pezzo avversario compatibile con quella linea, cioè una torre o donna su rank/file oppure un alfiere o donna su diagonale, crea un arco dal pezzo slider avversario al pezzo potenzialmente inchiodato. La relazione non enumera le case intermedie e non viene rappresentata come una sequenza di archi lungo l'intera linea: è una relazione diretta tra il pezzo che causa il pin e il pezzo inchiodato.

Anche `check_line` va letto secondo la semantica effettivamente implementata. Quando il lato al tratto è sotto scacco, vengono individuati gli attaccanti del suo re e viene creato un arco da ciascun checker alla casa del re. In caso di doppio scacco, possono quindi essere creati più archi con questa componente attiva. Il nome della feature non implica che tutte le case geometriche della linea di scacco siano esplicitamente rappresentate: la rappresentazione codifica il rapporto checker–king.

Se la stessa coppia ordinata di case soddisfa più relazioni, il grafo mantiene un unico arco e fonde le componenti tramite massimo elemento per elemento. Per esempio, una mossa legale che cattura un pezzo avversario può avere vettore

$$[1, 1, 0, 0, 0],$$

come nel caso concettuale $D2 \to E3$, dove l'arco rappresenta sia una mossa legale sia un attacco. Questa scelta evita duplicati strutturali e rende ogni arco una relazione multi-label tra due case.

Dopo la costruzione, gli archi vengono ordinati deterministicamente rispetto alla coppia sorgente-destinazione. La struttura finale `edge_index` ha forma $[2, E]$, dove E è il numero di archi della posizione, mentre `edge_attr` ha forma $[E, 5]$. Il valore

di E non è fisso: dipende dalla posizione, dal numero di mosse legali e dalle relazioni tattiche presenti.

4.4 Feature globali del grafo

Oltre alle feature locali di nodi e archi, la rappresentazione include un vettore di feature globali che descrive lo stato complessivo della posizione. Tali feature non sono associate a una singola casa o a una singola relazione, ma alla board nel suo insieme. Il layout è riportato nella Tabella 4.3.

Indice	Feature	Formula / codifica
0	side_to_move	1.0 se muove il Bianco, 0.0 se muove il Nero
1	is_check	1.0 se il lato al tratto è sotto scacco, altrimenti 0.0
2	fullmove_number	$\min(\text{fullmove}, 200)/200$
3	halfmove_clock	$\min(\text{halfmove}, 100)/100$

Tabella 4.3: Feature globali associate alla posizione scacchistica.

La prima feature codifica il lato al tratto; la seconda indica se il lato al tratto è attualmente sotto scacco. Le ultime due normalizzano i contatori FEN: il numero di mossa viene troncato superiormente a 200 e diviso per 200, mentre l'halfmove clock viene troncato a 100 e diviso per 100. In questo modo i contatori rimangono in un intervallo controllato.

Le feature globali descrivono solo la posizione. Non includono il rating del puzzle, la profondità Mate-in-N, l'identificativo del puzzle, i temi Lichess, la mossa target, la soluzione o eventuali informazioni temporali. Questi elementi appartengono alla label o ai metadati, non al vettore globale della posizione.

Nel codice corrente il vettore globale viene costruito come tensore di forma $[1, 4]$, in modo che il batching produca una matrice $[B, 4]$ per batch di B grafi.

4.5 PyTorch Geometric Data Object

Le componenti descritte nelle sezioni precedenti vengono aggregate in un oggetto `torch_geometric.data.Data`. Tale oggetto contiene sia la struttura del grafo sia le feature necessarie alla rappresentazione della posizione. La Tabella 4.4 distingue tra feature di input, struttura del grafo, label e metadati.

Le feature di input vere e proprie sono `x`, `edge_attr` e `global_features`, insieme alla struttura definita da `edge_index`. La label numerica `y` è ottenuta codificando la `TargetMove` tramite il vocabolario delle mosse discusso nel Capitolo 3. Se la mossa target non è presente nel vocabolario, non è possibile produrre una label numerica valida per i modelli a vocabolario fisso.

Campo	Shape / tipo	Ruolo	Categoria
`x`	$[64, 15]$ float	feature dei nodi	input
`edge_index`	$[2, E]$ long	archi diretti	struttura
`edge_attr`	$[E, 5]$ float	feature degli archi	input
`global_features`	$[1, 4]$ float	stato globale della posizione	input
`y`	scalare long	indice della mossa target	label
`fen`	stringa	posizione rappresentata	metadato
`target_move`	stringa UCI	mossa target originale	metadato / label testuale
`puzzle_id`	stringa	identificativo del puzzle	metadato
`rating`	scalare float	rating del puzzle	metadato
`mate_depth`	scalare long	profondità Mate-in-N	metadato
`source_row_index`	scalare long	riga sorgente nello split	metadato

Tabella 4.4: Campi principali dell'oggetto PyTorch Geometric usato per rappresentare una posizione.

I campi `fen` e `target_move` mantengono rispettivamente la posizione e la mossa target nel loro formato testuale originale. Gli altri campi, come `puzzle_id`, `rating` e `mate_depth`, permettono di conservare il collegamento con il record di origine e con i metadati descrittivi del puzzle. Nella pipeline corrente di generazione dei grafi viene inoltre aggiunto `source_row_index`, utile per ricondurre il grafo alla riga dello split tabellare da cui proviene.

La quantità E di archi varia con la posizione: ogni grafo ha sempre 64 nodi, ma il numero di relazioni scacchistiche rappresentate dipende da mosse legali, attacchi, difese, pin e stato di scacco. Questa variabilità è gestita naturalmente dal formato sparso di PyTorch Geometric tramite `edge_index`.

4.6 Generazione e persistenza del dataset a grafo

La trasformazione descritta nelle sezioni precedenti viene applicata sistematicamente alle righe degli split introdotti nel Capitolo 3. Per ciascun record, la FEN viene validata come posizione scacchistica, la mossa target viene verificata rispetto al vocabolario disponibile e viene costruito il grafo corrispondente. Una volta creato l'oggetto `Data`, vengono associati i metadati del puzzle, come identificativo, rating, profondità e indice della riga sorgente.

La gestione delle mosse fuori vocabolario è parte della generazione del dataset grafico. Se una `TargetMove` non è presente nel dizionario di codifica, il campione non può ricevere una label numerica coerente e viene conteggiato come OOV. Nella configurazione corrente questi esempi vengono saltati durante la generazione dei grafi. Errori di FEN o altre anomalie di conversione sono conteggiati separatamente, in modo da rendere tracciabile la differenza tra righe CSV e grafi effettivamente generati.

Il formato corrente previsto per la persistenza è sharded: i grafi vengono salvati in shard, ciascuno contenente un numero limitato di esempi, e un manifest descrive il dataset generato. Il manifest riporta lo schema delle feature, con dimensione 15 per i nodi, 5 per gli archi e 4 per le feature globali, oltre alle statistiche di generazione dei singoli split. Questo formato consente di rappresentare dataset di dimensione elevata senza dover caricare tutti i grafi in un unico oggetto monolitico.

La pipeline mantiene anche compatibilità con un formato legacy basato su file monolitici per ciascuno split. Tale supporto è rilevante perché nella working copy locale sono presenti artefatti legacy, mentre il formato sharded con manifest rappresenta la forma corrente prevista dalla pipeline. Questa differenza riguarda la persistenza e il caricamento dei grafi, non il layout concettuale della rappresentazione a grafo descritto nelle sezioni precedenti.

Capitolo 5

Architetture dei modelli

5.1 Panoramica della famiglia di modelli

I capitoli precedenti hanno definito il percorso che porta dai puzzle originali alla costruzione di esempi supervisionati e, successivamente, alla loro rappresentazione come grafi scacchistici. Il Capitolo 5 descrive i modelli che consumano tale rappresentazione e producono una predizione sulla mossa da giocare. Tutte le varianti considerate condividono la stessa impostazione generale del problema: a partire da una posizione, rappresentata come grafo, il modello deve assegnare un punteggio alla mossa corretta. Cambia invece il modo in cui lo spazio delle possibili mosse viene rappresentato e vincolato.

La famiglia di modelli segue una progressione architetturale abbastanza naturale. Il Modello A costituisce la baseline iniziale: la predizione della mossa viene trattata come un problema di classificazione su un vocabolario globale di mosse UCI. Il Modello A2 mantiene la stessa architettura, ma introduce un vincolo di legalità applicato ai logits prodotti dal classificatore. Il Modello A3 modifica invece il paradigma di output: invece di produrre un vettore su tutte le classi del vocabolario, assegna direttamente uno score a ciascuna mossa legale disponibile nella posizione corrente. Il Modello B estende A3 aggiungendo informazione temporale a livello di grafo. Infine, il Modello A4 usa A3 come primo stadio congelato e rivaluta un sottoinsieme di candidate considerando anche la posizione risultante dopo l'applicazione della mossa.

Si distinguono quindi due paradigmi principali. Nel primo, adottato da A e A2, lo spazio di output è fisso: ogni classe corrisponde a una mossa del vocabolario globale, indipendentemente dalla posizione corrente. Nel secondo, adottato da A3, B e A4, l'output è definito rispetto a un insieme variabile di candidate legali. Questa differenza è strutturale: nei modelli a vocabolario fisso la legalità della mossa deve essere gestita separatamente, mentre nei modelli candidate-based essa entra

direttamente nella definizione dello spazio di decisione.

La denominazione A1, presente nella documentazione storica del progetto, non corrisponde a un'architettura autonoma: indica una modalità di inferenza basata sulla selezione della migliore mossa legale a partire dai logits del Modello A. Per questo motivo non viene trattata come modello separato nel seguito del capitolo.

5.2 Encoder Graph Attention condiviso

Le varianti principali condividono un encoder Graph Attention che opera sulla rappresentazione a grafo introdotta nel Capitolo 4. L'input è quindi costituito dalle feature dei nodi, dagli archi diretti, dagli attributi degli archi e dalle feature globali della posizione. Il capitolo precedente ha descritto nel dettaglio il significato scacchistico di tali componenti; qui interessa invece il modo in cui esse vengono trasformate in embedding utilizzabili dai moduli di predizione.

Il primo layer dell'encoder è una convoluzione Graph Attention configurata come

$$\mathrm{GATConv}(15, 32, \mathrm{heads} = 4, \mathrm{concat} = \mathrm{True}, \mathrm{edge_dim} = 5).$$

Il valore 15 corrisponde alla dimensione delle feature di ciascun nodo, mentre edge_dim = 5 indica che gli attributi degli archi vengono effettivamente usati dal layer di attenzione. Poiché le quattro teste vengono concatenate, l'uscita del primo layer ha dimensione $32 \times 4 = 128$ per nodo. A questa trasformazione seguono una non linearità ELU e un dropout.

Il secondo layer mantiene la stessa dimensione finale degli embedding:

$$\mathrm{GATConv}(128, 32, \mathrm{heads} = 4, \mathrm{concat} = \mathrm{True}, \mathrm{edge_dim} = 5),$$

seguito nuovamente da ELU. Al termine dell'encoder, ogni casa della scacchiera è quindi rappresentata da un embedding di dimensione 128. Tali embedding vengono aggregati tramite global mean pooling, producendo per ciascun grafo un vettore di dimensione 128. Il vettore pooled viene infine concatenato con le quattro feature globali della posizione, ottenendo un contesto complessivo di dimensione

$$128 + 4 = 132.$$

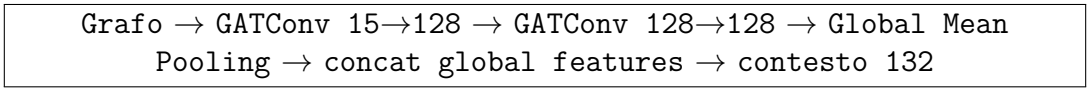


Figura 5.1: Schema dell'encoder Graph Attention condiviso dalle principali varianti architetturali.

Questo blocco viene usato direttamente nei modelli A, A2 e A3 sulla posizione originale. Il Modello B eredita lo stesso encoder e concatena al contesto della posizione un ulteriore vettore temporale. Il Modello A4, invece, utilizza un encoder analogo per rappresentare le posizioni risultanti dopo l'applicazione delle mosse candidate.

5.3 Modelli a vocabolario fisso: A e A2

I primi due modelli trattano la predizione della mossa come classificazione su uno spazio fisso di classi. Ogni classe rappresenta una mossa UCI appartenente al vocabolario costruito a partire dai dati di training. In questo paradigma, la dimensione dell'output non dipende dalla posizione corrente: il modello produce sempre un vettore di logits con una componente per ciascuna mossa del vocabolario.

5.3.1 Modello A: classificatore sul vocabolario globale

Il Modello A è la baseline no-timing basata sull'encoder Graph Attention condiviso. Dopo i due layer GAT e il pooling globale, la posizione è rappresentata dal contesto di dimensione 132. Tale vettore viene passato a un classificatore multilayer composto da una trasformazione lineare verso 128 dimensioni, una non linearità ELU, dropout e una trasformazione lineare finale verso lo spazio delle classi:

$$132 \rightarrow 128 \rightarrow \text{num_classes}.$$

Nelle configurazioni ufficiali dei modelli e dei checkpoint, il valore di num_classes è pari a 1786. Questo numero va interpretato come la dimensione configurata dello spazio di output per gli artefatti ufficiali, non come valore universalmente coerente con ogni artefatto locale del dataset. Come discusso nel Capitolo 3, infatti, esiste una discrepanza tra alcune risorse versionate del vocabolario e gli split CSV locali; per questo motivo il valore 1786 viene usato qui esclusivamente per descrivere l'architettura ufficiale dei modelli a vocabolario fisso.

L'output del Modello A ha forma $[B, \text{num_classes}]$, dove B è il numero di grafi nel batch. Ogni riga contiene i logits associati alle mosse del vocabolario. La proprietà strutturale più importante di questa formulazione è che lo spazio delle classi è globale: il modello non riceve in input l'insieme delle mosse legali della posizione e non incorpora nativamente il vincolo di legalità. Di conseguenza, dal punto di vista architetturale, può assegnare punteggi elevati anche a mosse che non sono legali nella posizione corrente.

Con le dimensioni ufficiali, il Modello A contiene 268.026 parametri addestrabili.

5.3.2 Modello A2: classificatore con mascheramento delle mosse illegali

Il Modello A2 non introduce una nuova Graph Neural Network. L'encoder, il pooling, il classificatore finale e il numero di parametri sono gli stessi del Modello A. La differenza riguarda il modo in cui vengono utilizzati i logits prodotti dal classificatore globale.

Per ciascuna posizione vengono enumerate le mosse legali. Tali mosse sono quindi associate, quando presenti, alle rispettive classi nel vocabolario globale. I logits delle classi non corrispondenti a mosse legali vengono sostituiti con un valore fortemente negativo, pari a -10^4 nell'implementazione. In questo modo, pur mantenendo un vettore di output definito sul vocabolario globale, la selezione della mossa viene confinata alle classi compatibili con la legalità della posizione corrente.

A2 va quindi interpretato come una variante legal-masked del classificatore A, non come un backbone architetturale indipendente. Il modello conserva lo stesso limite generale del vocabolario fisso, ma aggiunge un vincolo esplicito sulle classi ammissibili a valle della produzione dei logits. Anche in questo caso il numero di parametri addestrabili è 268.026.

5.4 Scoring delle mosse legali: Modello A3

Il Modello A3 introduce il cambiamento architetturale più rilevante rispetto ad A e A2. Invece di produrre un vettore di logits su un vocabolario globale, il modello riceve l'insieme delle mosse legali disponibili nella posizione corrente e assegna uno score scalare a ciascuna candidata. Lo spazio di output diventa quindi variabile e dipende direttamente dalla posizione.

La posizione viene codificata con lo stesso encoder Graph Attention descritto in precedenza. Il risultato è costituito dagli embedding dei nodi, ciascuno di dimensione 128, e dal contesto globale della posizione, di dimensione 132. Una mossa candidata viene rappresentata combinando quattro componenti: l'embedding del nodo sorgente, l'embedding del nodo destinazione, il contesto della posizione e un embedding della promozione.

Formalmente, per una mossa candidata m, il vettore usato dallo scorer può essere scritto come

$$h_m = [h_{\text{src}} \mid h_{\text{dst}} \mid h_G \mid h_{\text{promo}}],$$

dove h_{src} e h_{dst} hanno dimensione 128, h_G ha dimensione 132 e h_{promo} ha dimensione 8. La dimensione complessiva del vettore candidato è quindi

$$128 + 128 + 132 + 8 = 396.$$

La componente di promozione distingue cinque casi: nessuna promozione, promozione a donna, promozione a torre, promozione ad alfiere e promozione a cavallo. Tali casi non sono codificati come scalari grezzi, ma tramite un embedding apprendibile di dimensione 8. In assenza di promozione viene usato un identificativo dedicato, così che anche le mosse ordinarie abbiano una rappresentazione omogenea.

Il vettore h_m viene passato a uno scorer multilayer:

$$396 \to 128 \to 1,$$

con ELU e dropout tra i due layer lineari. L'uscita è uno score scalare per la candidata. Nel batching, le candidate di più posizioni vengono concatenate in una lista piatta; un vettore di puntatori, comunemente indicato come `candidate_ptr`, consente di ricostruire quali score appartengano alla stessa posizione. Questo dettaglio è necessario perché il numero di mosse legali varia da un grafo all'altro.

La conseguenza principale del paradigma A3 è che la legalità non viene applicata come correzione esterna a un vocabolario globale, ma entra nella definizione stessa delle candidate valutate. Il modello non produce logits per mosse non legali nella posizione corrente. Con le dimensioni ufficiali, A3 contiene 71.593 parametri addestrabili.

5.5 Modello B e informazione temporale

Il Modello B estende A3 aggiungendo informazione temporale a livello di grafo. Mantiene quindi lo stesso paradigma candidate-based: la posizione viene codificata con l'encoder Graph Attention, le mosse legali vengono rappresentate tramite nodo sorgente, nodo destinazione e promozione, e lo scorer finale produce uno score per ciascuna candidata.

La differenza è l'aggiunta di tre feature temporali associate al grafo: il tempo della mossa precedente, il tempo della decisione corrente e un indicatore binario che segnala la natura sintetica del tempo. Queste quantità sono trattate come feature globali aggiuntive, non come attributi di nodo o di arco. La loro generazione e normalizzazione appartengono al Capitolo 3; qui interessa solo il modo in cui vengono integrate nell'architettura.

Le tre feature temporali vengono elaborate da un piccolo encoder:

$$3 \to 16,$$

realizzato tramite un layer lineare, ELU e dropout. Il vettore risultante, di dimensione 16, viene concatenato al contesto della posizione prodotto da A3. Il contesto

globale passa quindi da 132 a

$$132 + 16 = 148$$

dimensioni.

Di conseguenza, anche il vettore candidato viene ampliato. Rimangono invariati l'embedding del nodo sorgente, l'embedding del nodo destinazione e l'embedding della promozione, ma il contesto della posizione include ora anche la componente temporale:

$$128 + 128 + 148 + 8 = 412.$$

Lo scorer finale ha quindi struttura

$$412 \to 128 \to 1,$$

con ELU e dropout tra le trasformazioni lineari.

L'output di B mantiene lo stesso formato concettuale di A3: uno score scalare per ciascuna mossa candidata legale. Il modello deve quindi essere letto come estensione locale di A3 con un encoder temporale compatto, non come implementazione diretta della libreria esterna TimeGNN. Con le dimensioni ufficiali, B contiene 73.705 parametri addestrabili.

5.6 Reranking delle posizioni post-mossa: Modello A4

Il Modello A4 introduce una struttura a due stadi. Nel primo stadio, A3 viene utilizzato come modello congelato per assegnare uno score alle mosse legali della posizione originale. Da questi score vengono selezionate le Top-5 candidate. Questa selezione non è un risultato sperimentale, ma una scelta architetturale e configurativa: A4 non rivaluta tutte le mosse legali, bensì un sottoinsieme ristretto proposto da A3.

Nel secondo stadio, ciascuna candidata selezionata viene applicata alla posizione corrente. Si ottiene così una nuova posizione, corrispondente allo stato della scacchiera dopo la mossa. Tale posizione post-mossa viene rappresentata nuovamente come grafo e codificata da un encoder GAT dedicato. L'encoder post-move segue la stessa struttura generale del backbone: due layer GAT con attributi degli archi, ELU, dropout, global mean pooling e concatenazione con le feature globali della posizione risultante. Il contesto post-move prodotto ha dimensione 132.

La rappresentazione usata dal reranker combina tre gruppi di feature. Il primo è il vettore candidato prodotto nello stile di A3 prima dello scorer, di dimensione 396. Il secondo è il contesto del grafo post-mossa, di dimensione 132. Il terzo è composto da due feature scalari derivate dagli score di A3: lo score grezzo della candidata e

lo score centrato rispetto al gruppo Top-5. La dimensione complessiva del vettore elaborato da A4 è quindi

$$396 + 132 + 2 = 530.$$

Il reranker finale è uno scorer multilayer:

$$530 \to 128 \to 1,$$

con ELU e dropout tra i due layer lineari. L'uscita è uno score scalare per ciascuna delle candidate Top-5. A differenza di A3, che valuta la mossa guardando la posizione originale e la coppia sorgente-destinazione, A4 dispone anche di informazione esplicita sulla posizione che risulterebbe dopo l'applicazione della candidata.

Dal punto di vista parametrico, A4 contiene 88.705 parametri addestrabili. Questo valore include l'encoder post-move e lo scorer del reranker, ma non i parametri di A3, che rimangono congelati. In inferenza, tuttavia, il sistema coinvolge anche A3 come stadio di retrieval; considerando A3 congelato, il totale dei parametri utilizzati è 160.298, di cui 71.593 appartengono al modello A3.

5.7 Confronto architetturale

La Tabella 5.1 riassume le differenze strutturali tra le varianti. La tabella ha funzione descrittiva: non confronta prestazioni, ma mette in evidenza il diverso modo in cui ciascun modello definisce lo spazio di decisione e incorpora vincoli o informazioni aggiuntive.

Modello	Paradigma di output	Legalità	Timing	Post-move	Parametri
A	vocabolario globale	non nativa	no	no	268.026
A2	vocabolario globale	maschera sui logits	no	no	268.026
A3	candidate legali	nativa	no	no	71.593
B	candidate legali	nativa	sì	no	73.705
A4	Top-5 candidate A3	vincolata da A3	no	sì	88.705

Tabella 5.1: Confronto architetturale tra le varianti della famiglia di modelli.

Nel caso di A4, il numero riportato indica i parametri addestrabili del reranker. Il modello utilizza anche A3 come componente congelata per produrre le candidate iniziali, ma tali parametri non vengono ottimizzati insieme al secondo stadio.

La progressione va dal vincolo di legalità di A2 allo scoring delle mosse legali di A3. Il Modello B aggiunge il timing, mentre A4 valuta le posizioni successive alle candidate. Il capitolo seguente definisce il confronto sperimentale.

Capitolo 6

Metodologia sperimentale

6.1 Disegno sperimentale

Gli esperimenti valutano la predizione della prima mossa risolutiva a partire dal grafo della posizione del solutore. Dati, rappresentazione e architetture sono definiti rispettivamente nei Capitoli 3, 4 e 5; questo capitolo descrive come i sistemi vengono confrontati. Le configurazioni ufficiali, gli iperparametri e i checkpoint sono riportati nell'Appendice A.

Le domande di ricerca richiedono due confronti principali. Per RQ1 il Modello B con timing è confrontato con Qwen 3.5 4B e 9B, separatamente per profondità di matto sul test Lichess. YACPDB non rientra in RQ1 perché non possiede gli ingressi temporali richiesti da B. Per RQ2 si confrontano A3 e B sugli stessi 8.610 grafi Lichess: i due modelli condividono lo scoring delle mosse legali e differiscono per l'integrazione del timing sintetico.

Gli altri confronti chiariscono il contributo delle scelte architetturali. A, A1 e A2 isolano gli effetti della classificazione globale e della legalità; A3 introduce lo scoring contestuale delle mosse legali; A4 misura l'effetto del reranking post-mossa sulle Top-5 candidate recuperate da A3. A1 è esclusivamente una modalità inferenziale del checkpoint A e non viene trattato come modello addestrato separatamente.

6.2 Addestramento e selezione dei checkpoint

Addestramento, validazione e test restano separati. Le configurazioni ufficiali fissano seed 42, ottimizzatore Adam, learning rate 5×10^{-4}, weight decay 10^{-4}, batch size 128, dropout 0,30 e mixed precision. L'early stopping usa solo la validazione; il test non partecipa alla selezione. A4 monitora la Top-1 end-to-end di validazione, mentre gli altri modelli monitorano la loss di validazione.

Le analisi finali usano checkpoint congelati e identificati nel manifesto del progetto. Nuovi addestramenti sono separati dagli artefatti ufficiali e non sostituiscono implicitamente i pesi di riferimento. A4 dipende dal checkpoint A3 congelato: A3 recupera le candidate, mentre vengono addestrati soltanto l'encoder post-mossa e lo scorer del reranker.

6.3 Protocollo di valutazione Lichess

La valutazione neurale principale usa i 8.610 grafi generati dal CSV di test dopo l'esclusione dei 10 target fuori vocabolario. Per ogni esempio, i sistemi producono un ordinamento delle mosse nel proprio spazio di uscita. A e A2 lavorano sul vocabolario globale, A1 filtra le predizioni di A, mentre A3 e B ordinano le mosse legali della posizione. A4 riceve soltanto le Top-5 candidate di A3 e ne modifica l'ordine; la sua Top-1 end-to-end include quindi sia il recupero sia il reranking.

L'ablazione del timing valuta B nella configurazione ufficiale, con timing neutralizzato e con valori temporali e indicatore azzerati. Tutte le configurazioni sono confrontate con A3 sugli stessi esempi. Il disegno consente di attribuire le differenze alla variante implementata, senza estenderle a forme di timing non osservate.

6.4 Protocollo dei modelli linguistici

Qwen 3.5 4B e Qwen 3.5 9B sono eseguiti localmente con prompt, parametri di generazione e parser congelati. Ogni richiesta contiene la FEN e richiede una sola mossa UCI. Il parser rigoroso `strict_uci_v1` accetta soltanto l'output previsto dal protocollo; la classificazione finale distingue risposta corretta, mossa legale errata, mossa illegale, fallimento di parsing ed errore di esecuzione.

Il parser rilassato è una diagnostica post-hoc e non sostituisce i risultati primari. Anche le prove GPT-OSS con problemi di protocollo sono escluse dal confronto ufficiale. Il prompt esatto e i dettagli operativi sono conservati nell'Appendice E; questa scelta permette di interpretare il risultato come prestazione del sistema completo nel protocollo fissato.

6.5 Benchmark esterno

Il benchmark `yacpdb_classic_v1` contiene 200 composizioni, 20 per profondità da #1 a #10, ed è separato dai dati Lichess. La valutazione usa la politica congelata delle chiavi accettate. Popeye verifica indipendentemente 193 posizioni: 188 hanno una chiave unica e 5 chiavi multiple; 7 verifiche terminano per timeout e mantengono la chiave sorgente YACPDB. Un timeout non equivale a una composizione errata.

Sul benchmark vengono valutati A3, A4 e i due modelli Qwen. B è indicato come non disponibile perché i dati non contengono il timing richiesto; non vengono generati valori sintetici ad hoc. Il benchmark è usato per studiare il trasferimento a una distribuzione esterna, non per selezionare modelli o iperparametri. L'intera procedura è specificata nell'Appendice D.

6.6 Metriche e analisi statistica

L'accuratezza Top-k è la percentuale di esempi nei quali il target compare tra le prime k predizioni. Se r_i è il rango del target per l'esempio i,

$$\text{Top-k} = \frac{100}{N} \sum_{i=1}^{N} \mathbb{I}(r_i \leq k). \tag{6.1}$$

Quando disponibile, il rango medio descrive anche la qualità dell'ordinamento oltre la soglia Top-k. Gli intervalli dell'accuratezza Top-1 sono calcolati con il metodo di Wilson al 95%. Per A3–B e A3–A4, quando le predizioni sono allineate per esempio, si usa il test esatto bilaterale di McNemar, formulato come test binomiale sui casi discordanti. L'effetto del timing è espresso anche come

$$\Delta_{\text{Top1}} = \text{Top1}_B - \text{Top1}_{A3}. \tag{6.2}$$

Le analisi selezionano esplicitamente gli artefatti ufficiali validi e separano risultati primari, diagnostiche, smoke test ed esecuzioni infrastrutturali non valide. Questa provenienza è riepilogata nell'Appendice F.

6.7 Limiti del protocollo

Il compito valuta una sola mossa e non la soluzione autonoma completa del puzzle. Il timing è sintetico, condizionato dal rating e disponibile soltanto su Lichess. A4 non può recuperare target assenti dalle Top-5 di A3. Il protocollo LLM misura una specifica combinazione di modelli, prompt e parser. Infine, i gruppi YACPDB contengono solo 20 esempi per profondità: le stime per singola categoria hanno intervalli ampi e non sostengono confronti fini. Questi limiti sono ripresi nel Capitolo 8 durante l'interpretazione dei risultati.

Capitolo 7

Risultati

7.1 Panoramica delle valutazioni

Questa sezione presenta i risultati degli esperimenti documentati nella repository GraphMate, utilizzando gli artefatti ufficiali già disponibili. L'analisi comprende il benchmark Lichess, il confronto controllato tra il modello senza timing A3 e la variante temporale B, la valutazione del reranking post-mossa introdotto da A4, il confronto con i modelli linguistici Qwen 3.5 4B e Qwen 3.5 9B e, infine, la valutazione su composizioni scacchistiche classiche provenienti da YACPDB.

Le tabelle riportano le metriche registrate negli artefatti dell'analisi finale. Per il benchmark Lichess, i risultati principali si riferiscono a 8 610 esempi effettivamente valutati. Per il benchmark classico, il campione comprende 200 composizioni. È importante distinguere questi due contesti: Lichess costituisce il benchmark principale, mentre YACPDB viene utilizzato per osservare il comportamento dei modelli su una distribuzione esterna.

L'analisi finale include 12 esperimenti ufficiali e mantiene separati i risultati primari dagli artefatti diagnostici o non validi. In particolare, i risultati dei modelli linguistici sono valutati secondo il protocollo strict UCI; i tentativi falliti per problemi infrastrutturali e i risultati ottenuti con il parser relaxed non vengono utilizzati per sostituire le misure principali.

7.2 Risultati sul benchmark Lichess

La Tabella 7.1 riassume le prestazioni dei modelli sul test set Lichess. L'accuratezza Top-1 indica la percentuale di esempi nei quali la mossa corretta è classificata al primo posto; Top-3 e Top-5 indicano invece la percentuale di esempi in cui la soluzione compare rispettivamente tra le prime tre o cinque alternative.

Tabella 7.1: Risultati principali sul benchmark Lichess ($N = 8\,610$).

Modello	Top-1 (%)	Top-3 (%)	Top-5 (%)	Rango medio
A	39,62	55,81	62,75	4,0115
A1	49,81	70,50	79,56	–
A2	49,26	71,50	81,17	3,7220
A3	67,56	85,66	91,86	2,2243
B	65,98	84,84	91,37	2,3029
A4	85,41	–	–	–

I risultati mostrano un miglioramento progressivo passando dalla classificazione globale delle mosse del modello A alle configurazioni che considerano esplicitamente la legalità delle mosse. Il modello A raggiunge un'accuratezza Top-1 del 39,62%, mentre la modalità inferenziale A1, che applica un filtro per selezionare la migliore mossa legale, raggiunge il 49,81%. Il modello A2, che introduce una maschera di legalità nella classificazione, ottiene il 49,26% di Top-1 e il valore Top-5 più elevato tra A, A1 e A2, pari all'81,17%.

Il modello A3, che assegna punteggi alle mosse legali candidate, ottiene un'accuratezza Top-1 del 67,56%, con Top-3 pari all'85,66% e Top-5 pari al 91,86%. Rispetto al modello A, il miglioramento assoluto di Top-1 è pari a circa 27,94 punti percentuali. Il risultato indica che, nel benchmark considerato, la formulazione basata sullo scoring delle mosse legali è più efficace delle configurazioni precedenti basate sul vocabolario globale.

Il modello A4 raggiunge il valore Top-1 più alto tra i modelli a grafo riportati nella tabella, pari all'85,41%. Questo risultato va interpretato tenendo conto della sua struttura: A4 riordina le prime cinque mosse recuperate da A3 utilizzando una valutazione GNN delle posizioni successive alle mosse candidate. La metrica Top-1 di A4 è quindi il risultato di una pipeline di recupero e reranking e non di una classificazione indipendente su tutte le mosse legali. Nell'artefatto riepilogativo finale sono riportati per A4 soltanto i valori Top-1; per questo motivo le altre metriche non vengono indicate.

A1, inoltre, è una modalità inferenziale applicata al modello A e non dispone di un checkpoint autonomo. Il suo risultato è pertanto utile per misurare l'effetto del filtro di legalità in inferenza, ma non deve essere interpretato come il risultato di un'architettura addestrata separatamente.

7.3 Contributo delle informazioni temporali

L'esperimento relativo al timing confronta il modello A3, privo di feature temporali, con il modello B, che utilizza informazioni temporali sintetiche. Per mantenere

il confronto controllato, entrambe le configurazioni sono valutate sugli stessi 8 610 esempi.

Tabella 7.2: Confronto tra A3 e le configurazioni di ablazione del timing su Lichess.

Configurazione	Top-1 (%)	Top-3 (%)	Top-5 (%)	Δ Top-1 (p.p.)
A3, senza timing	67,56	85,66	91,86	0,00
B, timing sintetico	65,98	84,84	91,37	-1,58
B, timing neutralizzato	66,10	84,84	91,37	-1,46
B, timing e flag azzerati	66,10	84,85	91,37	-1,46

Il modello B raggiunge una Top-1 del 65,98%, inferiore di 1,58 punti percentuali rispetto al 67,56% ottenuto da A3. La differenza è coerente anche con le metriche Top-3 e Top-5, che diminuiscono rispettivamente di circa 0,81 e 0,49 punti percentuali. Il rango medio della mossa corretta passa da 2,2243 per A3 a 2,3029 per B, indicando un peggioramento anche nell'ordinamento medio delle mosse target.

Le configurazioni di ablazione confermano che la presenza delle feature temporali sintetiche non produce un miglioramento rispetto alla baseline senza timing. Con il timing neutralizzato o con i valori temporali e il relativo flag azzerati, la Top-1 si attesta al 66,10%, ancora inferiore a quella di A3.

Il confronto appaiato tra A3 e B viene valutato mediante il test esatto bilaterale di McNemar. Tra i casi discordanti, B corregge 321 esempi nei quali A3 era errato, mentre A3 è corretto e B errato in 457 esempi. Il test restituisce $p \approx 1{,}22 \times 10^{-6}$, indicando una differenza statisticamente significativa tra le predizioni appaiate, nella direzione sfavorevole a B. L'effetto osservato è pari a $-1{,}58$ punti percentuali di Top-1.

La conclusione deve rimanere circoscritta all'esperimento implementato: il segnale temporale sintetico utilizzato in GraphMate non migliora le prestazioni predittive rispetto alla baseline A3. Poiché i tempi sono sintetici e condizionati dal rating, questo risultato non dimostra che informazioni temporali reali siano inutili; indica soltanto che la codifica temporale valutata non apporta un beneficio misurabile in questo protocollo.

7.4 Prestazioni in funzione della profondità del matto

Per approfondire i risultati Lichess, le prestazioni Top-1 sono state analizzate separatamente per profondità del problema. La Tabella 7.3 riporta i valori di A3, B, A4 e delle due baseline LLM.

Tabella 7.3: Accuratezza Top-1 per profondità del matto sul benchmark Lichess.

Profondità	N	A3 (%)	B (%)	A4 (%)	Qwen 4B / 9B (%)
Mate-in-1	1 995	79,55	78,15	93,93	0,10 / 0,10
Mate-in-2	1 999	69,78	68,33	88,19	0,05 / 0,05
Mate-in-3	1 999	66,73	65,48	84,59	0,15 / 0,10
Mate-in-4	1 999	58,73	56,73	78,39	0,00 / 0,20
Mate-in-5	618	52,91	50,65	74,27	0,00 / 0,16

Per A3, B e A4 si osserva una diminuzione dell'accuratezza al crescere della profondità del matto. A3 passa dal 79,55% nei problemi mate-in-1 al 52,91% nei mate-in-5; B passa dal 78,15% al 50,65%, mentre A4 passa dal 93,93% al 74,27%. La tendenza è coerente con una maggiore difficoltà di previsione al crescere della profondità della soluzione, pur senza costituire da sola una misura universale della difficoltà scacchistica.

In tutte le cinque categorie, B presenta una Top-1 inferiore a quella di A3. Anche A4 mantiene la Top-1 più elevata tra i modelli a grafo riportati. Le accuratezze delle baseline Qwen restano invece molto basse secondo il protocollo strict UCI, con valori compresi tra lo 0% e lo 0,20% nelle singole categorie.

I gruppi mate-in-1 fino a mate-in-4 contengono circa 2 000 esempi ciascuno, mentre mate-in-5 ne contiene 618. Il confronto tra profondità deve quindi tenere conto della diversa numerosità dei gruppi, soprattutto per l'ultima categoria.

7.5 Analisi per fascia di rating

La repository riporta anche un'analisi delle prestazioni di A3 e A4 suddivisa per fasce di rating associate ai problemi. I risultati sono riportati nella Tabella 7.4.

Tabella 7.4: Accuratezza Top-1 per fascia di rating sul benchmark Lichess.

Fascia di rating	N	A3 (%)	A4 (%)
< 1200	3 879	82,06	95,54
1200–1599	2 176	65,85	86,90
1600–1999	1 564	51,34	75,83
2000–2399	811	43,16	60,42
2400+	180	26,67	45,00

Per entrambi i modelli, la Top-1 diminuisce all'aumentare della fascia di rating. A3 passa dall'82,06% nella fascia inferiore a 1200 al 26,67% nella fascia 2400+, mentre A4 passa dal 95,54% al 45,00%. A4 mantiene prestazioni superiori ad A3 in tutte le fasce considerate.

Questa analisi descrive un'associazione tra la fascia di rating dei problemi e la prestazione dei modelli; non consente, da sola, di attribuire causalmente il calo a una singola caratteristica delle posizioni. La fascia 2400+ comprende inoltre soltanto 180 esempi, per cui la relativa stima è basata su un campione sensibilmente più piccolo rispetto alle fasce inferiori.

7.6 Confronto con le baseline LLM su Lichess

Il confronto con i modelli Qwen 3.5 4B e Qwen 3.5 9B viene effettuato mediante generazione di una singola mossa e parsing strict in formato UCI. I risultati complessivi sono riportati nella Tabella 7.5.

Tabella 7.5: Confronto tra modelli a grafo e baseline LLM su Lichess ($N = 8\,610$).

Modello	Top-1 (%)	Intervallo di confidenza Wilson al 95%
A3	67,5610	[66,5645; 68,5418]
B	65,9814	[64,9738; 66,9748]
A4	85,4123	[84,6509; 86,1421]
Qwen 3.5 4B strict	0,0697	[0,0319; 0,1520]
Qwen 3.5 9B strict	0,1161	[0,0631; 0,2137]

Nel benchmark considerato, i modelli a grafo ottengono accuratezze Top-1 nettamente superiori alle due baseline Qwen secondo il protocollo di valutazione adottato. Il risultato deve tuttavia essere letto alla luce della composizione degli errori dei modelli linguistici: la valutazione strict richiede che la singola mossa generata sia interpretabile nel formato previsto e corrisponda a una mossa corretta.

Per Qwen 3.5 4B, soltanto 6 esempi su 8 610 risultano corretti. La repository registra 130 mosse legali ma errate, 5 894 mosse illegali e 2 580 fallimenti di parsing. Per Qwen 3.5 9B, gli esempi corretti sono 10, a fronte di 413 mosse legali ma errate, 2 899 mosse illegali e 5 288 fallimenti di parsing. Per entrambi i modelli non vengono registrati errori di esecuzione nel riepilogo ufficiale.

Questa distribuzione mostra che le prestazioni strict delle baseline LLM sono influenzate in misura rilevante dalla produzione di output non conformi e di mosse illegali. Non è quindi corretto interpretare la sola Top-1 come una misura completa delle capacità di ragionamento scacchistico dei modelli linguistici: essa misura il successo complessivo nel compito specifico, che include sia la generazione di un output interpretabile sia la correttezza della mossa.

L'analisi primaria non sostituisce questi risultati con quelli ottenuti tramite parsing relaxed, che restano esclusivamente diagnostici. Anche i tentativi sperimentali GPT-OSS sono esclusi dal confronto principale, poiché la repository li conserva co-

me evidenza di problemi di protocollo e generazione, non come risultati comparabili validi.

7.7 Risultati sul benchmark esterno YACPDB

Il benchmark YACPDB comprende 200 composizioni classiche, con 20 problemi per ciascuna profondità di matto da uno a dieci. La Tabella 7.6 riporta le prestazioni complessive dei modelli valutati.

Tabella 7.6: Risultati sul benchmark esterno YACPDB ($N = 200$).

Modello	Top-1 (%)	Top-3 (%)	Top-5 (%)	Soluzioni Top-1
A3	11,0	23,5	34,5	22
A4	15,5	29,5	34,5	31
Qwen 3.5 4B strict	0,0	–	–	0
Qwen 3.5 9B strict	0,5	–	–	1
B	–	–	–	–

Sul benchmark esterno, A4 raggiunge una Top-1 del 15,5%, rispetto all'11,0% di A3. Anche Top-3 aumenta dal 23,5% al 29,5%, mentre Top-5 rimane invariata al 34,5%. In termini assoluti, A4 identifica correttamente la mossa accettata in 31 composizioni, contro le 22 di A3.

Il confronto appaiato A3–A4, effettuato con il test esatto bilaterale di McNemar, considera 23 casi discordanti: in 16 casi A4 è corretto e A3 errato, mentre in 7 casi accade il contrario. Il valore $p \approx 0{,}0931$ non raggiunge la soglia convenzionale di significatività del 5%. Il miglioramento descrittivo di A4 è quindi evidente nelle metriche osservate, ma il test appaiato non fornisce evidenza sufficiente per dichiarare statisticamente significativa la differenza al livello del 5%.

Per il benchmark classico, Qwen 3.5 4B non individua alcuna mossa accettata, mentre Qwen 3.5 9B ne individua una su 200. Anche in questo caso, i conteggi degli errori mostrano una quota elevata di output non conformi o illegali. Il modello B non viene valutato su YACPDB, poiché il benchmark non contiene gli input temporali richiesti dalla sua configurazione; il risultato è pertanto indicato come non disponibile e non viene sostituito con valori sintetici.

I risultati per singola profondità devono essere interpretati con cautela, perché ciascun gruppo contiene soltanto 20 composizioni. In queste condizioni, ogni singolo problema corrisponde a cinque punti percentuali di Top-1 e gli intervalli di confidenza sono ampi. Le differenze tra profondità non costituiscono quindi stime precise del comportamento generale dei modelli a ciascun livello di matto.

7.8 Generalizzazione tra Lichess e YACPDB

Il confronto tra il benchmark principale e quello esterno permette di osservare la variazione delle prestazioni quando i modelli vengono applicati a composizioni provenienti da una distribuzione differente. La Tabella 7.7 riassume la Top-1 nei due benchmark.

Tabella 7.7: Confronto della Top-1 tra Lichess e YACPDB.

Modello	Lichess (%)	YACPDB (%)	Differenza (p.p.)
A3	67,56	11,00	-56,56
A4	85,41	15,50	-69,91
Qwen 3.5 4B strict	0,0697	0,0000	-0,0697
Qwen 3.5 9B strict	0,1161	0,5000	+0,3839

Per i modelli a grafo, il passaggio da Lichess a YACPDB è associato a una riduzione marcata della Top-1: da 67,56% a 11,0% per A3 e da 85,41% a 15,5% per A4. Questo risultato evidenzia una difficoltà nel trasferire le prestazioni ottenute sul dataset principale alle composizioni classiche esterne.

La differenza non deve essere interpretata come una prova che YACPDB sia semplicemente più difficile in senso assoluto. I due benchmark differiscono per provenienza, costruzione e distribuzione dei problemi; i risultati indicano pertanto un effetto di generalizzazione esterna e un possibile cambiamento di distribuzione (*distribution shift*). Per le baseline Qwen, le percentuali sono prossime allo zero in entrambi i benchmark e la variazione assoluta osservata non consente di trarre conclusioni generali sulla capacità di generalizzazione dei modelli linguistici.

7.9 Sintesi delle misure

Le misure principali fissano tre osservazioni per la discussione successiva. Su Lichess A3 supera i modelli a vocabolario globale, mentre A4 ottiene la Top-1 più alta. Nel confronto controllato, B rimane al di sotto di A3. Sul benchmark esterno, A4 mantiene un vantaggio descrittivo su A3, ma entrambi mostrano valori molto inferiori a quelli Lichess. Il Capitolo 8 interpreta questi risultati, ne distingue evidenze e ipotesi e ne discute i limiti.

Capitolo 8

Discussione

8.1 Vincoli scacchistici e spazio decisionale

La progressione da A ad A3 mostra che la rappresentazione della decisione è importante almeno quanto l'encoder condiviso. Il Modello A può assegnare probabilità a classi incompatibili con la posizione corrente; A1 e A2 riducono questo problema filtrando o mascherando le mosse illegali, mentre A3 costruisce direttamente il confronto tra le sole mosse legali disponibili. Sul test Lichess, il passaggio dal 39,62% di A al 49,81% di A1 isola il beneficio del vincolo in inferenza senza introdurre un nuovo checkpoint. A3 raggiunge il 67,56%, suggerendo che formulare l'apprendimento come ranking contestuale delle candidate sia più efficace del solo filtro applicato a un vocabolario globale.

Questa evidenza non implica che la legalità esaurisca la difficoltà scacchistica. Il generatore di mosse elimina azioni impossibili, ma il modello deve ancora distinguere la continuazione tatticamente corretta dalle alternative legali. Il risultato indica piuttosto che incorporare un vincolo certo del dominio riduce uno spazio decisionale inutilmente ampio e allinea meglio l'obiettivo di addestramento con quello di valutazione.

8.2 Reranking post-mossa e limite di recupero

A4 aggiunge una seconda prospettiva: non valuta soltanto la posizione corrente e la coppia origine–destinazione, ma costruisce il grafo ottenuto dopo ogni candidata e ne usa il contesto per il reranking. L'aumento della Top-1 dal 67,56% di A3 all'85,41% su Lichess è coerente con l'utilità di osservare le conseguenze immediate della mossa. Sul benchmark classico il vantaggio descrittivo è più contenuto, dall'11,0% al 15,5%, e il test appaiato non raggiunge la soglia convenzionale del 5% ($p \approx 0{,}0931$); il campione di 200 problemi limita la precisione della stima.

Il risultato di A4 è condizionato dallo stadio di recupero. Il reranker riceve solo le cinque migliori candidate di A3: se la mossa accettata non compare in questo insieme, A4 non può selezionarla. La Top-5 di A3 costituisce quindi un tetto di recupero end-to-end; su YACPDB essa è pari al 34,5%. A4 migliora l'ordinamento all'interno dell'insieme disponibile, ma non risolve gli errori di recupero a monte. Questa distinzione evita di attribuire al solo encoder post-mossa l'intera prestazione della pipeline a due stadi.

8.3 Timing e confronto con i modelli linguistici

Nel confronto controllato il Modello B perde 1,58 punti percentuali di Top-1 rispetto ad A3; i casi discordanti e il test esatto di McNemar indicano che la differenza non è spiegata soltanto da oscillazioni casuali nel campione valutato. Le ablazioni neutralizzate restano inferiori ad A3. L'evidenza supporta quindi una conclusione circoscritta: la codifica temporale sintetica implementata non migliora lo scorer di mosse legali.

Non segue che il tempo di riflessione sia irrilevante in generale. I valori usati sono generati deterministicamente e condizionati dal rating; non misurano il tempo impiegato da una persona per risolvere quello specifico puzzle. È plausibile, ma non dimostrato dagli esperimenti, che il segnale non aggiunga informazione indipendente o che l'encoder non ne sfrutti la forma. Verificare queste ipotesi richiederebbe tempi reali allineati ai puzzle o un diverso disegno sperimentale.

Nel protocollo rigoroso UCI, entrambi i modelli Qwen ottengono accuratezze molto inferiori ai modelli a grafo a ogni profondità Lichess; RQ1 riceve dunque una risposta uniforme nei cinque gruppi disponibili. Il risultato dimostra l'efficacia dei sistemi completi sotto il protocollo congelato, non una superiorita generale delle GNN su ogni uso possibile dei modelli linguistici. Prompt singolo, generazione di una sola mossa, assenza di strumenti scacchistici e parser rigoroso delimitano il confronto. L'elevata quota di output non interpretabili o illegali è parte della prestazione end-to-end misurata, ma non equivale da sola a una misura pura del ragionamento scacchistico interno.

8.4 Generalizzazione e limiti sperimentali

Il calo di A3 e A4 da Lichess a YACPDB segnala un cambio di distribuzione: le composizioni classiche differiscono dai puzzle estratti da partite per provenienza, costruzione e profondità. I dati non provano che YACPDB sia intrinsecamente più difficile; mostrano invece che le prestazioni sulla distribuzione principale non si

trasferiscono integralmente. Il campione di 200 problemi, 20 per profondità, rende inoltre instabili i confronti fini tra categorie.

La portata del lavoro è circoscritta da ulteriori limiti. GraphMate predice la prima mossa risolutiva, non l'intera variante. Il timing sintetico manca in YACPDB, quindi RQ2 resta limitata a Lichess; A4 dipende dal recupero Top-5 di A3. Le baseline linguistiche coprono un protocollo locale, non ogni modello, prompt o integrazione con motori. Checkpoint e budget sono quelli delle configurazioni ufficiali: i risultati descrivono esecuzioni congelate, non limiti teorici.

Capitolo 9

Conclusioni

GraphMate ha sviluppato una pipeline riproducibile per rappresentare posizioni di matto come grafi scacchistici e predire la prima mossa risolutiva. La progressione dei modelli mostra che l'integrazione dei vincoli del dominio produce i miglioramenti più rilevanti: il filtro di legalità migliora la baseline a vocabolario globale, lo scoring contestuale delle mosse legali porta A3 al 67,56% di Top-1 su Lichess e il reranking delle posizioni successive alla mossa porta A4 all'85,41% nel medesimo benchmark.

Risposta a RQ1. Sul benchmark Lichess, il Modello B con timing supera entrambe le baseline Qwen sottoposte al protocollo rigoroso UCI in tutte le profondità disponibili, da mate-in-1 a mate-in-5. La conclusione riguarda il confronto end-to-end congelato; non estende automaticamente il risultato ad altri modelli linguistici, prompt o protocolli assistiti da strumenti.

Risposta a RQ2. Il timing sintetico implementato non migliora la prestazione rispetto alla baseline A3 senza timing. La Top-1 di B è pari al 65,98%, contro il 67,56% di A3, con una differenza di (-1,58) punti percentuali confermata dal confronto appaiato. Il risultato non dimostra che tempi reali e allineati ai puzzle siano inutili; delimita l'esito alla codifica sintetica valutata.

La valutazione esterna completa il quadro: A4 conserva un vantaggio descrittivo su A3 nelle composizioni YACPDB, ma le accuratezze scendono sensibilmente rispetto a Lichess e la differenza appaiata non è statisticamente significativa al 5%. Il cambio di distribuzione, la dimensione ridotta del benchmark, il limite Top-5 del reranker, il timing sintetico e la predizione di una sola mossa sono i principali vincoli interpretativi.

Nel complesso, gli esperimenti sostengono una conclusione precisa: per il compito e i dati considerati, esplicitare legalità, candidate e conseguenze immediate delle mosse è più utile dell'aggiunta del segnale temporale sintetico adottato. Allo stesso tempo, il divario sul benchmark esterno mostra che accuratezza interna e generalizzazione non coincidono e devono essere valutate separatamente.

Bibliografia

[1] Lichess. *lichess.org open database*. URL: https://database.lichess.org/ (visitato il 15/10/2026).

[2] Thomas N. Kipf e Max Welling. «Semi-Supervised Classification with Graph Convolutional Networks». In: *International Conference on Learning Representations*. 2017. URL: https://arxiv.org/abs/1609.02907.

[3] Petar Veličković et al. «Graph Attention Networks». In: *International Conference on Learning Representations*. 2018. URL: https://arxiv.org/abs/1710.10903.

[4] Nancy Xu, Chrysoula Kosma e Michalis Vazirgiannis. «TimeGNN: Temporal Dynamic Graph Learning for Time Series Forecasting». In: *arXiv preprint arXiv:2307.14680* (2023). URL: https://arxiv.org/abs/2307.14680.

Appendici

Appendice A

Configurazioni ufficiali, iperparametri e protocollo di addestramento

Questa appendice riporta le configurazioni ufficiali dei modelli usate per i risultati sperimentali di GraphMate. Il suo scopo è distinguere tre oggetti che possono essere facilmente confusi: una *configurazione ufficiale*, che specifica architettura, sorgente dei dati, ottimizzatore e politica di addestramento; un *checkpoint congelato*, che contiene i pesi addestrati selezionati tramite criteri di validazione; e un *artefatto di valutazione*, che registra le metriche prodotte da un checkpoint con un valutatore fissato. L'Appendice B descrive la rappresentazione a grafo, l'Appendice C descrive la costruzione dei dataset e l'Appendice F documenta i workflow di riproducibilità a livello di repository.

Model A, Model A2, Model A3, Model B e Model A4 sono modelli addestrati con configurazioni JSON ufficiali e hash dei checkpoint. A1 non è un modello addestrato separatamente. È una variante solo di inferenza di Model A che applica una maschera sulle mosse legali al momento della predizione; di conseguenza non possiede una configurazione, uno stato dell'ottimizzatore o un checkpoint indipendente.

A.1 Politica di addestramento comune

I modelli neurali addestrati usano una struttura comune di addestramento, salvo dove indicato diversamente nella Tabella A.1. Le configurazioni ufficiali fissano il seed casuale a 42, usano Adam con learning rate 5×10^{-4}, weight decay 10^{-4}, batch size 128, dropout 0.30 e addestramento mixed precision. L'early stopping usa solo la prestazione di validazione; l'insieme di test mantenuto separato non è usato per la selezione del modello. Model A4 è l'unica eccezione nell'obiettivo dello

scheduler: monitora la Top-1 end-to-end di validazione per il reranking e quindi usa uno scheduler di massimizzazione, mentre gli altri modelli addestrati monitorano la loss di validazione e usano una minimizzazione.

Tabella A.1: Impostazioni ufficiali comuni di addestramento. I valori si applicano a Model A, Model A2, Model A3, Model B e Model A4 salvo diversa indicazione nella colonna delle note.

Impostazione	Valore	Note
Seed	42	Usato dalle configurazioni ufficiali.
Ottimizzatore	Adam	Stessa famiglia di ottimizzatori per i modelli addestrati.
Learning rate	5×10^{-4}	Learning rate iniziale dell'ottimizzatore.
Weight decay	1×10^{-4}	Weight decay di Adam.
Batch size	128	Batch size ufficiale di addestramento.
Epoche massime	300	L'early stopping può fermare prima l'addestramento.
Pazienza early stopping	12	Solo metrica di validazione; test escluso dalla selezione.
Min delta early stopping	0	Non è richiesto un margine positivo di miglioramento.
Dropout	0.30	Probabilità di dropout dell'architettura.
Scheduler	ReduceLROnPlateau	Factor 0.5, patience 3, learning rate minimo 10^{-6}.
Modalità scheduler	min tranne A4 max	A/A2/A3/B monitorano la loss di validazione; A4 monitora la Top-1 end-to-end di validazione.
AMP	abilitato	Mixed precision nell'addestramento ufficiale.
Worker	0	`num_workers=0`; worker persistenti disabilitati.
Pinned memory	abilitata	Trasferimenti non bloccanti abilitati.

A.2 Sintesi delle architetture dei modelli

La Tabella A.2 sintetizza le differenze architetturali. Model A e Model A2 sono classificatori a vocabolario globale sulle 1,786 mosse target presenti nel vocabolario

di addestramento. Model A2 aggiunge una maschera sulle mosse legali durante lo scoring. Model A3 sostituisce lo spazio di uscita globale con le mosse candidate legali disponibili nella posizione corrente. Model B mantiene lo scorer delle mosse candidate legali, ma arricchisce il contesto del grafo con feature di timing sintetico provenienti dal dataset di timing descritto nell'Appendice C. Model A4 congela Model A3, recupera le sue Top-5 mosse candidate legali, applica ciascuna mossa candidata, codifica la posizione risultante con una GNN separata sui grafi successivi alla mossa e addestra solo il reranker.

Tabella A.2: Sintesi delle architetture ufficiali dei modelli. I conteggi dei parametri sono mostrati solo dove verificati da artefatti finali.

Modello	Ruolo	Spazio di uscita	Dimensioni principali	Parametri verificati
A	Baseline senza timing a vocabolario globale	1,786 classi	nodo 15, arco 5, globale 4, nascosto 128, classifier nascosto 128	Non affermato
A1	Variante inference-only con maschera legale di A	Checkpoint A con filtro legale	Nessuna configurazione di addestramento separata	Nessun checkpoint
A2	Modello a vocabolario globale mascherato	1,786 classi con maschera legale	Stesse dimensioni di base di A	Non affermato
A3	Scorer delle mosse candidate legali	Candidate legali variabili	candidato $128 + 128 + 132 + 8 = 396$	71,593
B	Scorer delle mosse candidate legali con timing	Candidate legali variabili	ingresso timing 3, nascosto timing 16, candidato 412	73,705
A4	Reranker GNN sui grafi successivi alla mossa	Top-5 candidate del Model A3 congelato	contesto post-mossa 132, feature score 2, candidato 530	88,705 addestrabili; 160,298 totali in inferenza

Le dimensioni nella Tabella A.2 derivano direttamente dall'implementazione. L'encoder del grafo produce $32 \times 4 = 128$ canali nascosti per nodo. Il global mean pooling produce un embedding del grafo a 128 dimensioni, concatenato con quattro feature globali per formare un contesto del grafo a 132 dimensioni. Model A3 rappresenta quindi ciascuna mossa candidata con embedding del nodo sorgente, embedding del nodo destinazione, contesto del grafo ed embedding della promozione:

$$128 + 128 + 132 + 8 = 396.$$

Model B aggiunge un embedding di timing a 16 dimensioni al contesto del grafo:

$$128 + 128 + (132 + 16) + 8 = 412.$$

Model A4 riusa la rappresentazione candidata congelata di A3 a 396 dimensioni e aggiunge un contesto post-mossa a 132 dimensioni e due feature scalari dello score

A3:

$$396 + 132 + 2 = 530.$$

A.3 Inventario dei checkpoint congelati

L'inventario ufficiale dei checkpoint è riportato nella Tabella A.3. La tabella riporta solo gli hash disponibili dalla configurazione del repository e dal manifesto dei checkpoint. I conteggi dei parametri sono indicati solo quando verificati da artefatti finali di addestramento: Model A3, Model B e Model A4. Questa appendice non afferma conteggi dei parametri per Model A o Model A2 perché la provenienza disponibile in questo passaggio non fornisce un conteggio finale verificato per quei due checkpoint.

Tabella A.3: Percorsi e hash SHA-256 dei checkpoint congelati ufficiali. A1 è omesso perché è inference-only e non ha un checkpoint separato.

Modello	Percorso del checkpoint	SHA-256
A	`checkpoints/model_a/best.pt`	`1a72d0b6f675b50c829d76c63343e7016a569c00...`
A2	`checkpoints/model_a2/best.pt`	`20a76ae5648ded7f8fe3eb835eddd5ec0a87c904...`
A3	`checkpoints/model_a3/best.pt`	`4efec653a451da7585f3663847c8dc5caaa8ebff...`
B	`checkpoints/model_b/best.pt`	`b8bbad2420ce301582ad08377b1c80f56609513a...`
A4	`checkpoints/model_a4/best.pt`	`0cb73acf70487efa5c93f715a5a60c0aa09d6479...`

A.4 Note per modello

Model A è la baseline senza timing a vocabolario globale. Il suo classificatore mappa il contesto del grafo a 132 dimensioni su 1,786 classi di uscita. L'esecuzione ufficiale è stata proseguita da un budget iniziale di 150 epoche fino a un massimo di 300 epoche perché l'andamento di validazione giustificava la prosecuzione; l'insieme di test è rimasto escluso dalla decisione di selezione.

Model A2 conserva l'architettura e la politica di addestramento di Model A, ma valuta le mosse tramite una maschera legale. Costituisce quindi un confronto controllato con la baseline a vocabolario globale non mascherata, mantenendo la stessa rappresentazione dei dati e lo stesso vocabolario target.

Model A3 è lo scorer principale delle mosse candidate legali senza timing. Assegna uno score alle mosse legali generate dallo stato della scacchiera invece che all'intero vocabolario di addestramento. Il suo conteggio verificato di parametri addestrabili è 71,593, e il checkpoint selezionato tramite validazione è il riferimento senza timing usato da Model A4.

Model B è lo scorer delle mosse candidate legali con timing. Richiede il dataset PyG con timing e non effettua fallback silenzioso al dataset a grafo senza timing. L'encoder del timing mappa tre input di timing in un embedding a 16 dimensioni

prima della concatenazione con il contesto del grafo. Il suo conteggio verificato di parametri addestrabili è 73,705.

Model A4 è un reranker GNN sui grafi successivi alla mossa applicato alle Top-5 candidate del Model A3 congelato. Il checkpoint congelato di A3 è una dipendenza di inferenza, non una componente addestrabile di A4. Lo scheduler di A4 monitora la Top-1 end-to-end di validazione, quindi ReduceLROnPlateau usa la modalità “max”. L’artefatto finale riporta 88,705 parametri addestrabili di A4 e 160,298 parametri totali in inferenza quando si include la dipendenza A3 congelata.

Appendice B

Rappresentazione a grafo e schema delle feature

Questa appendice specifica la rappresentazione da scacchiera a grafo usata dai modelli neurali di GraphMate. È una specifica implementativa, non una spiegazione generale delle reti neurali su grafi. L'Appendice A indica quali modelli consumano ciascuna rappresentazione, mentre l'Appendice C descrive come i file di grafo sono prodotti a partire dai dati sorgente.

B.1 Indicizzazione delle case e layout dei tensori

Ogni posizione è rappresentata come un grafo PyTorch Geometric con un nodo per ciascuna casa della scacchiera. I nodi sono ordinati secondo l'indice di casa di python-chess, da $a1 = 0$ fino a $h8 = 63$. La coordinata di riga è l'indice di traversa diviso per 7 e la coordinata di colonna è l'indice di colonna diviso per 7:

$$r(s) = \frac{\text{rank}(s)}{7}, \qquad c(s) = \frac{\text{file}(s)}{7}.$$

Pertanto la prima traversa ha coordinata di riga 0, l'ottava traversa ha coordinata di riga 1, la colonna a ha coordinata di colonna 0 e la colonna h ha coordinata di colonna 1. L'oggetto dati del grafo contiene la matrice dei nodi $x \in \mathbb{R}^{64\times 15}$, l'indice degli archi sparse COO `edge_index` $\in \mathbb{N}^{2\times E}$, la matrice degli archi `edge_attr` $\in \mathbb{R}^{E\times 5}$ e le feature globali `global_features` $\in \mathbb{R}^4$. Il dataset conserva inoltre campi di metadati, tra cui FEN, mossa target, identificatore del puzzle, rating, profondità di matto e indice della riga sorgente.

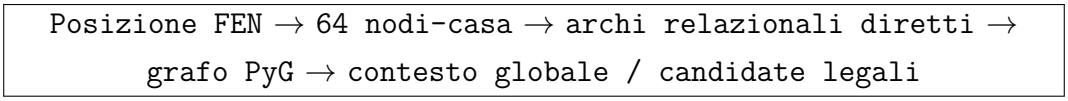


Figura B.1: Costruzione del grafo a livello concettuale. I canali esatti di nodi, archi e feature globali sono specificati nelle Tabelle B.1, B.2 e B.3.

B.2 Feature dei nodi

La Tabella B.1 riporta l'ordine esatto delle 15 dimensioni dei nodi. Le case vuote ricevono indicatori di tipo pezzo pari a zero, colore 0, occupazione 0, pin 0 e valore del pezzo 0, mantenendo però le coordinate normalizzate e le feature di conteggio degli attacchi. Il campo colore vale 1 per un pezzo bianco e 0 per un pezzo nero o per una casa vuota. La feature di mobilità legale conta le mosse legali che hanno origine dalla casa dopo aver valutato copie della scacchiera con il tratto impostato prima al Bianco e poi al Nero; è quindi un conteggio di mobilità per casa di origine, non un'etichetta di mossa legale per il target di addestramento. La feature di pezzo inchiodato è locale a un nodo occupato e non deve essere confusa con l'indicatore globale di scacco nella Tabella B.3.

Tabella B.1: Ordine delle feature dei nodi per ciascuno dei 64 nodi-casa.

Indice	Feature	Codifica
0–5	Tipo di pezzo	One-hot per pedone, cavallo, alfiere, torre, donna, re; tutti zero per case vuote.
6	Colore	1 per pezzo bianco; 0 per pezzo nero o casa vuota.
7	Occupata	1 se una casa contiene un pezzo; altrimenti 0.
8	Riga	$\text{rank}(s)/7$, con rank 0 sulla prima traversa.
9	Colonna	$\text{file}(s)/7$, con file 0 sulla colonna a.
10	Attaccata dal Bianco	Conteggio degli attaccanti bianchi della casa nello stato corrente.
11	Attaccata dal Nero	Conteggio degli attaccanti neri della casa nello stato corrente.
12	Mobilità legale	Conteggio delle mosse legali originate dalla casa su stati copiati con tratto al Bianco e al Nero.
13	Pezzo inchiodato	1 se il pezzo occupante è inchiodato al proprio re; 0 per pezzi non inchiodati o case vuote.
14	Valore del pezzo	Pedone 1, cavallo/alfiere 3, torre 5, donna 9, re 0, vuota 0.

B.3 Feature degli archi

Gli archi sono diretti e multi-etichetta. Se più relazioni si applicano alla stessa coppia ordinata di case, i cinque canali dell'arco sono aggregati tramite massimo componente per componente. La lista risultante degli archi è memorizzata in formato sparse COO. Gli archi di mossa legale puntano dall'origine alla destinazione per ogni mossa legale nella posizione. Gli archi di attacco e difesa puntano da una casa sorgente occupata a una casa bersaglio occupata attaccata dal pezzo sorgente;

il canale è attacco per colori opposti e difesa per lo stesso colore. Gli archi di inchiodatura sono generati lungo le linee del re dal pezzo scorrevole attaccante al pezzo inchiodato. Gli archi di linea di scacco sono generati solo quando il lato al tratto è sotto scacco e collegano l'attaccante che dà scacco al re; non enumerano tutte le case sulla linea geometrica.

Tabella B.2: Canali delle feature degli archi diretti. Relazioni multiple sulla stessa coppia diretta sono combinate tramite massimo componente per componente.

Indice	Canale	Semantica
0	Mossa legale	Arco origine-destinazione per ciascuna mossa legale nella posizione.
1	Attacco	La sorgente occupata attacca un bersaglio occupato di colore opposto.
2	Difesa	La sorgente occupata attacca un bersaglio occupato dello stesso colore.
3	Inchiodatura	Pezzo scorrevole attaccante verso pezzo inchiodato lungo la linea del re.
4	Linea di scacco	Attaccante che dà scacco verso re quando il lato al tratto è sotto scacco.

B.4 Feature globali

Le quattro feature globali nella Tabella B.3 codificano lato al tratto, stato di scacco e contatori di mossa con clipping. Le normalizzazioni dei contatori sono

$$\text{fullmove_norm} = \frac{\min(\text{fullmove}, 200)}{200}, \qquad \text{halfmove_norm} = \frac{\min(\text{halfmove}, 100)}{100}.$$

Queste feature sono concatenate all'embedding del grafo ottenuto con pooling e sono conservate anche nei grafi successivi alla mossa usati da Model A4.

Tabella B.3: Ordine delle feature globali del grafo.

Indice	Feature	Codifica
0	Lato al tratto	1 per Bianco al tratto, 0 per Nero al tratto.
1	Sotto scacco	1 se il lato al tratto è sotto scacco; altrimenti 0.
2	Fullmove	$\min(\text{fullmove}, 200)/200$.
3	Halfmove	$\min(\text{halfmove}, 100)/100$.

B.5 Rappresentazioni delle mosse candidate

I modelli basati su mosse candidate legali costruiscono un vettore candidato per ciascuna mossa legale. Il raggruppamento delle candidate è memorizzato con un vettore di puntatori, così che la loss sia calcolata sulle mosse legali appartenenti alla stessa posizione. Le promozioni usano un identificatore a cinque valori, con nessuna promozione mappata a 0 e donna, torre, alfiere e cavallo mappati rispettivamente a 1–4 prima dell'applicazione di un embedding di promozione a 8 dimensioni.

Model A3 concatena embedding della casa sorgente, embedding della casa destinazione, contesto del grafo ed embedding della promozione. Model B aggiunge un embedding di timing al contesto del grafo. Model A4 chiede prima al Model A3 congelato le Top-5 candidate, poi applica ciascuna candidata per produrre un grafo della posizione risultante, codifica tale grafo e concatena il contesto successivo alla mossa con la rappresentazione candidata congelata di A3 e due feature dello score A3. La Tabella B.4 riporta l'aritmetica verificata delle dimensioni.

Tabella B.4: Dimensioni dei vettori candidati per i modelli a mosse legali e di reranking.

Modello	Componenti	Dimensione
A3	nodo sorgente 128 + nodo destinazione 128 + contesto grafo 132 + embedding promozione 8	396
B	nodo sorgente 128 + nodo destinazione 128 + contesto grafo con timing 132 + 16 + embedding promozione 8	412
A4	candidato A3 congelato 396 + contesto post-mossa 132 + feature score A3 2	530

Appendice C

Pipeline dei dati, suddivisioni e timing sintetico

Questa appendice specifica la pipeline di trattamento dei dati usata per gli esperimenti finali di GraphMate. Il repository contiene materiale esplorativo aggiuntivo, incluso lavoro sul timing derivato da partite Lichess, ma gli esperimenti finali di Model A3, Model B e Model A4 si basano sulla pipeline dei puzzle Lichess descritta qui. Model B usa campi di timing sintetico associati a quei grafi di puzzle; non è addestrato su osservazioni reali dell'orologio di gioco.

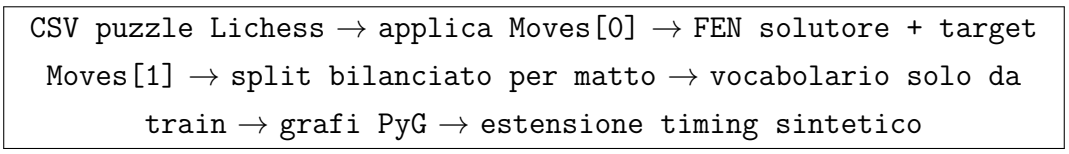


Figura C.1: Pipeline finale dei dati dei puzzle. Il timing esplorativo derivato da partite Lichess non fa parte della sorgente finale di addestramento per A3/B/A4.

C.1 Semantica dei puzzle Lichess

Il file sorgente pulito per il task finale sui puzzle è `data/processed/puzzles/mate_puzzles_clean.csv`. Nel formato dei puzzle Lichess, il campo grezzo `FEN` indica la posizione prima della mossa di preparazione dell'avversario. La fase di preprocessing registra quindi questa posizione come `OriginalFEN`, applica `Moves[0]` come mossa UCI legale di preparazione e usa la scacchiera risultante come FEN del solutore. L'etichetta target è `Moves[1]`, cioè la prima mossa attesa dal solutore nella posizione trasformata. Le righe sono scartate se la FEN originale non è valida, se la mossa di preparazione è malformata o illegale, oppure se la mossa target è malformata o illegale nella posizione del solutore.

Dopo correzione semantica e validazione, la pipeline conserva identificatore del puzzle, FEN originale, FEN del solutore, stringa completa delle mosse, mossa target,

rating, temi e profondità di matto. Il costruttore degli split finali bilancia il dataset per profondità di matto con al massimo 20,000 esempi per profondità, quindi costruisce gli insiemi di addestramento, validazione e test con rapporti 0.80, 0.10 e 0.10 usando random state 42 e stratificazione per profondità di matto.

C.2 Vocabolario delle mosse e conteggi dei grafi

Il vocabolario delle mosse è costruito solo dall'insieme di addestramento. Contiene 1,786 mosse target uniche su 68,958 esempi di addestramento. I target di validazione e test assenti dal vocabolario di addestramento sono trattati come fuori vocabolario (OOV) per il dataset a vocabolario globale e vengono saltati durante la generazione dei grafi PyG. La Tabella C.1 riporta sia il numero di righe CSV sia il numero di grafi generati, in modo che la distinzione tra dimensione dello split e numero di grafi utilizzabili sia esplicita.

Tabella C.1: Statistiche finali degli split dei puzzle Lichess. Le righe CSV sono quelle dei file di split; i grafi PyG escludono le mosse target fuori dal vocabolario costruito solo sull'addestramento.

Split	Righe CSV	Grafi PyG	OOV saltati	Shard	Note
Addestramento	68,958	68,958	0	69	Sorgente del vocabolario
Validazione	8,620	8,612	8	9	0.0928% OOV
Test	8,620	8,610	10	9	0.1160% OOV

Il dataset PyG generato usa lo schema di grafo dell'Appendice B. I grafi sono scritti in shard da 1,000 esempi per impostazione predefinita. Il loader sharded usa una piccola cache LRU degli shard e un sampler consapevole degli shard, in modo che l'addestramento non richieda di caricare l'intero dataset in memoria.

C.3 Dataset di timing sintetico

Model B usa un dataset PyG arricchito con timing, con la stessa appartenenza agli split e gli stessi conteggi di grafi del dataset PyG senza timing. Il timing è deterministico e sintetico. Per ciascun identificatore di puzzle e stream, una chiave stabile basata su blake2b mappa la tupla (seed, stream, puzzle id) a un valore pseudocasuale riproducibile; il seed globale è 42. Sia

$$d = \text{clip}\left(\frac{\text{rating} - 600}{2400 - 600}, 0, 1\right)$$

lo scalare di difficoltà dipendente dal rating. La mediana del tempo della mossa corrente è

$$m = 12.0(1 + 0.55d).$$

Con estrazioni normali standard z_c e z_p dal generatore stabile, i log-tempi della mossa corrente e della mossa precedente sono

$$\ell_c = \log(m) + 0.45 z_c,$$

$$\ell_p = 0.70\ell_c + 0.30\log(m) + 0.35 z_p.$$

I secondi sono ottenuti per esponenziazione e limitati all'intervallo $[0.5, 120.0]$. La normalizzazione è stimata solo sull'insieme di addestramento:

$$z(t) = \frac{\log(1+t) - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

dove il manifesto del repository registra $\mu_{\text{train}} = 2.7518570738318733$ e $\sigma_{\text{train}} = 0.4437502604800904$. La Tabella C.2 elenca i campi di timing memorizzati con ciascun grafo.

Tabella C.2: Campi di timing associati al dataset PyG di Model B. I valori normalizzati usano parametri log1p z-score stimati solo sull'addestramento.

Campo	Significato
`previous_move_time`	Tempo sintetico normalizzato della mossa precedente.
`original_move_time`	Tempo sintetico normalizzato della mossa corrente/originale.
`time_is_synthetic`	Indicatore pari a 1.0 per tutti gli esempi finali con timing.
`previous_move_time_seconds`	Tempo grezzo della mossa precedente in secondi, dopo clipping.
`original_move_time_seconds`	Tempo grezzo della mossa corrente/originale in secondi, dopo clipping.

Il flag di timing vale sempre 1.0 per il dataset con timing sintetico e il manifesto registra zero esempi con timing reale. I controlli contro la leakage sono quindi strutturali: il generatore sintetico è indicizzato dall'identificatore del puzzle invece che dall'uscita del modello, le statistiche di normalizzazione sono stimate solo sull'insieme di addestramento e le mosse target non sono usate per generare i valori di timing.

Appendice D

Costruzione e verifica del benchmark di composizioni classiche

Il benchmark di composizioni classiche valuta GraphMate sotto un cambio di distribuzione esterno: problemi di matto diretto composti provenienti da YACPDB invece di puzzle tattici Lichess. Il benchmark non è usato per addestramento, selezione dei checkpoint, selezione delle candidate o progettazione dei prompt. Il suo scopo è verificare se sistemi addestrati o valutati su puzzle di matto in stile Lichess si trasferiscono a posizioni di composizione curate, con provenienza e struttura differenti.

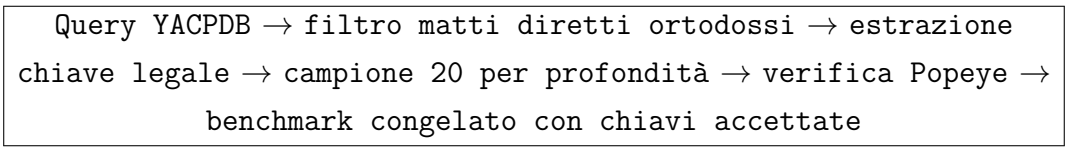


Figura D.1: Pipeline di costruzione e congelamento del benchmark di composizioni classiche.

D.1 Acquisizione ed eleggibilità

Le posizioni YACPDB sono state acquisite tramite l'endpoint gateway pubblico ma non documentato `https://yacpdb.org/gateway/ql`. Le query di acquisizione hanno usato `Stip("^#1$")` fino a `Stip("^#10$")` con richieste paginate. La dimensione osservata delle pagine era 100 e la scansione di disponibilità ha ispezionato fino a 200 record per profondità di matto. Poiché l'endpoint è pubblico ma non documentato, la riproducibilità si basa sulle risposte in cache, sui manifesti e sui fingerprint memorizzati con il benchmark, non sull'assunzione che il servizio rimanga stabile.

L'eleggibilità è intenzionalmente conservativa. L'importer conserva problemi ortodossi di matto diretto #1–#10 con pezzi supportati, posizioni strutturalmente identificabili e chiave iniziale legale. Scarta automatti, aiutomatti, stipulazioni di serie, condizioni fairy, gemelli, posizioni malformate, notazione della chiave non supportata, chiavi illegali, incompatibilità nei marcatori di cattura e chiavi iniziali ambigue o mancanti. La normalizzazione della posizione converte liste algebriche di pezzi supportate in FEN, normalizza i matti diretti con il Bianco al tratto e non inventa diritti di arrocco o en-passant. L'estrattore della chiave iniziale ignora gioco apparente, tentativi, confutazioni e semimosse successive della soluzione; il target del benchmark è la prima mossa risolutiva.

Tabella D.1: Scansione di disponibilità YACPDB dopo il filtraggio conservativo di eleggibilità. La scansione ha ispezionato fino a 200 record per profondità di matto.

Profondità	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10	Totale
Eleggibili puliti	53	186	191	189	193	181	132	120	126	169	1,540
Selezionati	20	20	20	20	20	20	20	20	20	20	200

D.2 Selezione, fingerprint e ciclo di congelamento

Dopo il filtraggio di eleggibilità, il pool candidato contiene 1,540 posizioni pulite sulle profondità #1–#10. La selezione ordina i candidati puliti per identificatore numerico del problema sorgente YACPDB, campiona 20 posizioni per profondità di matto con Python `random.Random(42)` e scrive le righe selezionate ordinate per profondità e identificatore numerico. Il benchmark risultante contiene 200 posizioni. I controlli di duplicazione riportano zero identificatori sorgente duplicati, zero duplicati FEN esatti e zero duplicati di posizione normalizzata. L'audit di contaminazione riporta zero sovrapposizioni con i dati dei puzzle Lichess usati per lo sviluppo dei modelli GraphMate.

Sono rilevanti due fingerprint. Il fingerprint del dataset candidato selezionato è:

bb1b2d7c3858e3b2ffad58fd561534acf1e29bb2321b4af52a57256398ec9a5a

Il fingerprint del ciclo di vita del benchmark congelato è:

57ff2b7c725a4ef802817581fac9430f8c6416a6de54dafcb1deb7b471716b60

Il valutatore verifica il fingerprint del congelamento, non soltanto il fingerprint candidato, prima di accettare un'esecuzione ufficiale. Questa distinzione evita che un dataset selezionato sia trattato come congelato prima che la politica di verifica e i metadati delle chiavi accettate siano fissati.

D.3 Verifica Popeye e chiavi accettate

Popeye è usato come verificatore indipendente di composizioni, non come modello, target di addestramento o baseline motore scacchistico. Il verificatore ufficiale è Popeye v4.103. Il primo passaggio ha usato un timeout di 300 secondi per posizione e ha verificato 183 delle 200 posizioni selezionate. Il secondo passaggio ha rieseguito i 17 timeout con un timeout di 1,200 secondi e ha verificato ulteriori 10 posizioni. Il benchmark consolidato ha quindi 193 posizioni verificate da Popeye: 188 con una chiave iniziale accettata unica e 5 con più chiavi iniziali accettate. Sette posizioni restano non risolte da Popeye per timeout. Questi sette casi non sono etichettati come composizioni errate; sono conservati in una categoria separata di timeout non verificato dalla sorgente per evitare di sostituire posizioni difficili dopo aver osservato informazioni sul tempo di risoluzione.

Tabella D.2: Stato della verifica Popeye per il benchmark congelato di composizioni classiche. Timeout significa verifica indipendente non risolta, non etichetta di composizione errata.

Stato	Conteggio	Note
Chiave iniziale unica verificata	188	Una chiave accettata.
Chiavi iniziali multiple verificate	5	Più chiavi accettate secondo la politica congelata.
Timeout non risolto	7	Conservato con categoria della chiave sorgente YACPDB.
Totale	200	Dimensione del benchmark congelato.

Lo scoring ufficiale accetta una predizione se appartiene all'insieme congelato delle chiavi accettate per la posizione. Per le posizioni con più chiavi verificate da Popeye, qualunque chiave iniziale accettata è corretta. Per le posizioni non risolte per timeout, la chiave sorgente YACPDB mantenuta è usata nella categoria timeout. Model B non è applicabile a questo benchmark perché le posizioni esterne di composizione non dispongono dei metadati di timing sintetico richiesti dal contratto di ingresso di Model B.

D.4 Risultati della valutazione sulle composizioni classiche

La Tabella D.3 riporta i risultati principali sul benchmark di composizioni classiche. ALL200 indica l'intero benchmark congelato. Popeye193 indica le 193 posizioni verificate indipendentemente da Popeye. La distinzione è metodologica, non un filtro

prestazionale post-hoc, perché tutte le 200 posizioni rimangono parte del benchmark congelato e le sette posizioni non risolte hanno una categoria di scoring esplicita.

Tabella D.3: Risultati principali sul benchmark di composizioni classiche secondo la politica congelata delle chiavi accettate. Le percentuali sono Top-1 salvo dove Top-3 o Top-5 è indicato esplicitamente.

Sistema	ALL200	Popeye193	Top-3	Top-5	Corrette
Model A3	11.0%	11.399%	23.5%	34.5%	22
Model A4	15.5%	15.026%	29.5%	34.5%	31
Qwen 3.5 4B rigoroso	0.0%	0.0%	–	–	0
Qwen 3.5 9B rigoroso	0.5%	0.518%	–	–	1
Model B	N/A	N/A	N/A	N/A	–

Model B: timing non disponibile. ALL200 e Popeye193 riportano la Top-1.

Tabella D.4: Accuratezza Top-1 sulle composizioni classiche per categoria di verifica.

Sistema	Unica $N = 188$	Multipla $N = 5$	Timeout $N = 7$
Model A3	10.106%	60.0%	0.0%
Model A4	13.298%	80.0%	28.571%
Qwen 3.5 4B rigoroso	0.0%	0.0%	0.0%
Qwen 3.5 9B rigoroso	0.532%	0.0%	0.0%

I risultati sulle composizioni classiche vanno interpretati come evidenza su una distribuzione esterna, non come sostituzione dell'insieme di test Lichess mantenuto separato. Model A4 migliora rispetto a Model A3 su ALL200, mentre le baseline Qwen con prompt rigoroso restano vicine a zero sotto il parser congelato descritto nell'Appendice E.

Appendice E

Protocollo di valutazione dei modelli linguistici

Questa appendice specifica il protocollo congelato di valutazione dei large language model usato in GraphMate. Il protocollo è volutamente ristretto: ciascun modello riceve una FEN e deve restituire una sola mossa legale in notazione UCI. Non è un benchmark di agente scacchistico interattivo e non fornisce nel prompt mosse legali, profondità di matto, rating del puzzle, temi, mosse target, linee del motore o predizioni di GraphMate.

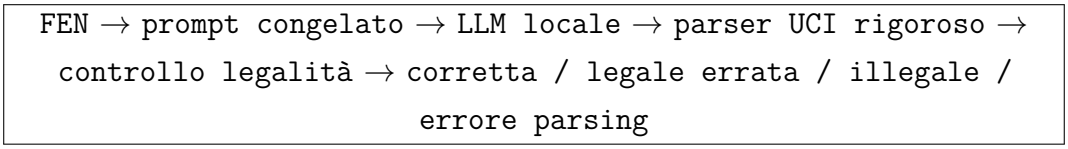


Figura E.1: Pipeline di valutazione LLM per il task next-move. Il parsing rilassato è diagnostico e non fa parte del percorso di scoring principale.

E.1 Identità del runtime e prompt

I modelli locali principali sono Qwen 3.5 4B e Qwen 3.5 9B serviti tramite Ollama. Il registro congelato riporta tag e digest esatti: Qwen 3.5 4B usa il tag `qwen3.5:4b` con digest 2a654d98e6fba55d452b7043684e9b57a947e393bbffa62485a7aac05ee4eefd; Qwen 3.5 9B usa il tag `qwen3.5:9b` con digest 6488c96fa5faab64bb65cbd30d42 89e20e6130ef535a93ef9a49f42eda893ea7. Entrambe le esecuzioni principali usano temperatura 0, `num_predict=16` e modalità thinking disabilitata quando supportata dalla configurazione del runtime.

La versione del prompt è `llm_chess_uci_v1`. Il prompt seguente è riprodotto verbatim perché costituisce il prompt congelato esatto usato negli esperimenti. Il system prompt congelato è:

```
You are solving a chess position.\nReturn exactly one legal
chess move for the side to move.\nDo not provide explanation
or analysis.
```

Lo user prompt congelato è:

```
FEN:\n{fen}\n\nReturn your move in UCI notation only.
```

L'hash del prompt registrato nell'audit delle appendici è 52d7d99ae114c426c41cf4 9b22b16e6038f2b4188e7ca81386783a3d51e52adc.

E.2 Parser rigoroso e tassonomia degli esiti

Il parser principale è `strict_uci_v1`. Rimuove spazi iniziali e finali dall'uscita del modello, richiede un singolo token che corrisponda a [a-h][1-8][a-h][1-8][qrbn]? e controlla la legalità con python-chess nella posizione fornita dal prompt. Una risposta che non rispetta la regola del token è un errore di parsing. Un token in forma UCI che non è legale nella posizione è una mossa illegale. Una mossa legale è considerata corretta solo se coincide con la mossa target congelata per il task next-move o con la mossa attesa del solutore per il passo corrente del task reference-line. Le altre mosse legali sono mosse legali errate.

La popolazione principale Lichess è costituita dagli 8,610 grafi di test generati dal manifesto PyG dei puzzle. Gli esiti rigorosi nella Tabella E.1 sono quindi percentuali su 8,610 esempi, non sulle 8,620 righe CSV originali.

Tabella E.1: Esiti rigorosi del task next-move Lichess per i modelli Qwen su 8,610 grafi di test generati.

Modello	Corrette	Legali errate	Illegali	Parsing	Esecuzione
Qwen 3.5 4B	6 (0.0697%)	130 (1.510%)	5,894 (68.455%)	2,580 (29.965%)	0
Qwen 3.5 9B	10 (0.116%)	413 (4.797%)	2,899 (33.670%)	5,288 (61.417%)	0

E.3 Parser rilassato diagnostico

Il parser rilassato è un artefatto diagnostico post-hoc, non un benchmark principale alternativo. Misura se alcuni fallimenti del contratto rigoroso di uscita contengano una mossa legale recuperabile in un'altra notazione comune o in forma testuale inglobata. Le affermazioni principali del report continuano a usare il parser rigoroso. La Tabella E.2 riporta i risultati diagnostici Lichess e il miglioramento rispetto alla Top-1 rigorosa in punti percentuali.

Tabella E.2: Diagnostiche del parser rilassato sul task next-move Lichess. Sono diagnostiche post-hoc, non punteggi del benchmark principale.

Modello	Corrette	Legali errate	Illegali	Parsing	Δ Top-1
Qwen 3.5 4B	10 (0.116%)	446	5,894	2,260	+0.046 pp
Qwen 3.5 9B	17 (0.197%)	1,023	2,899	4,671	+0.081 pp

E.4 Valutazione sulle composizioni classiche e reference-line

Lo stesso prompt, parser rigoroso e tassonomia di legalità sono usati per il benchmark di composizioni classiche nell'Appendice D. Qwen 3.5 4B ottiene 0 mosse corrette su ALL200 sotto il parser rigoroso. Qwen 3.5 9B ottiene 1 mossa corretta su ALL200, pari allo 0.5 percento su ALL200 e a circa 0.518 percento sul sottoinsieme Popeye193. Il benchmark registra separatamente mosse legali errate, mosse illegali, errori di parsing ed errori di esecuzione, così che i fallimenti del formato di uscita non siano confusi con i fallimenti di legalità scacchistica.

Per le diagnostiche reference-line a più semimosse, ogni mossa del solutore è richiesta indipendentemente dalla FEN corrente. Se il modello restituisce la mossa attesa del solutore, il valutatore applica quella mossa e poi applica la risposta attesa dell'avversario dalla linea di riferimento congelata. Se il modello sbaglia, produce una mossa illegale o non analizzabile, la linea termina. Questo protocollo impedisce ai prompt successivi di rivelare mosse future della soluzione.

E.5 Validità delle esecuzioni ed esclusioni

Le esecuzioni ufficiali eseguono controlli preliminari su raggiungibilità del runtime, digest esatto del modello, comportamento del parser, fingerprint del dataset, versione del prompt, versione del parser e assenza di leakage del target. Un tentativo di esecuzione fallito non è automaticamente un risultato scientifico. I fallimenti infrastrutturali sono marcati con metadati di esecuzione non valida ed esclusi dai riepiloghi di accuratezza. GPT-OSS 20B compare nell'inventario degli esperimenti come esecuzione diagnostica con fallimento di protocollo ed è escluso dal confronto principale di GraphMate. Questa gestione distingue identità del modello, tentativo di esecuzione del runtime e risultato valido del benchmark.

Appendice F

Riproducibilità e inventario degli artefatti

Questa appendice descrive come il repository GraphMate supporta la riproducibilità scientifica. Evita deliberatamente di ripetere ogni iperparametro, feature del grafo e regola di benchmark. Tali dettagli appartengono alle Appendici A–E. L'enfasi qui è posta sui confini degli artefatti, sui file destinati alla distribuzione pubblica e sulla differenza tra ispezione, valutazione di checkpoint congelati e riaddestramento.

F.1 Classi di artefatti

GraphMate separa configurazioni, checkpoint, risorse, dataset generati e uscite di analisi. Le configurazioni ufficiali definiscono architettura prevista e politica di addestramento. I checkpoint congelati memorizzano i pesi addestrati selezionati e sono identificati da hash SHA-256. Le risorse includono file ausiliari stabili, come il vocabolario delle mosse. Gli artefatti contengono report di addestramento, uscite di valutazione e riepiloghi dell'analisi finale. La Tabella F.1 sintetizza le principali classi di artefatti destinate alla riproducibilità.

Tabella F.1: Classi di artefatti del repository rilevanti per la riproducibilità.

Classe di artefatto	Esempi	Ruolo nella riproducibilità
Configurazioni ufficiali	`configs/model_*.json`	Architettura, root dei dati, ottimizzatore, scheduler e politica di addestramento.
Checkpoint congelati	`checkpoints/model_*/best.pt`	Pesi pretrained identificati da SHA-256.
Risorse	JSON del move encoder	Mappature ausiliarie stabili, come il vocabolario delle mosse costruito sull'addestramento.
Dataset generati	`data/pyg`, `data/pyg_puzzles_timing`	Ingressi a grafo e ingressi a grafo arricchiti con timing.
Artefatti di valutazione	report finali, JSON classici, uscite LLM	Uscite metriche e metadati delle esecuzioni.
Analisi finale	`generic_info/final_analysis`	Tabelle aggregate destinate al report e inventario.

F.2 Tre casi d'uso per la riproduzione

Il repository supporta tre diversi obiettivi di riproduzione, mostrati nella Tabella F.2. L'ispezione dei risultati pubblicati richiede soltanto gli artefatti e i manifesti usati dal report. La valutazione dei modelli pretrained congelati richiede i file dei checkpoint, gli hash esatti, i dataset a grafo e gli entry point di valutazione. Il riaddestramento usa le configurazioni ufficiali tramite `train_official`; è una riesecuzione scientifica con la stessa configurazione prevista, non una garanzia di uguaglianza bit-per-bit tra hardware, CUDA, PyTorch o versioni di PyTorch Geometric.

Tabella F.2: Tre casi d'uso distinti per la riproduzione.

Caso d'uso	Artefatti richiesti	Dichiarazione attesa
Ispezionare i risultati pubblicati	File di analisi finale, report, manifesti	Ricostruire tabelle e provenienza dei risultati riportati.
Valutare modelli congelati	Checkpoint, dataset a grafo, manifesti dei benchmark, valutatori	Ricalcolare le metriche per pesi addestrati fissati.
Riaddestrare configurazioni ufficiali	Codice sorgente, configurazioni, dati, ambiente di addestramento	Riprodurre la procedura prevista di addestramento, senza garanzia di pesi byte-identici.

F.3 Interfaccia ufficiale di addestramento

L'entry point ufficiale di addestramento è `src/cli/training/train_official.py`. Accetta `-model a`, `a2`, `a3`, `b` o `a4`, oppure un percorso esplicito di configurazione. La modalità `-dry-run` ispeziona e valida la configurazione selezionata senza avviare l'addestramento. Le uscite di riaddestramento sono indirizzate a `artifacts/retraining/<model>/<timestamp>/`; l'interfaccia rifiuta di scrivere nelle directory canoniche dei checkpoint. Questa regola di sicurezza protegge l'inventario dei checkpoint congelati da sovrascritture accidentali.

Le dipendenze dati specifiche per modello sono applicate dai loader. Model B richiede la root PyG con timing invece della root dei grafi senza timing. Model A4 richiede il checkpoint congelato di Model A3 e la cache delle candidate post-mossa di A4. Questi requisiti fanno parte della definizione del modello, non sono semplici comodità opzionali.

F.4 Riproduzione di checkpoint e valutazioni

Gli hash dei checkpoint congelati sono elencati nell'Appendice A. Gli script di valutazione devono verificare fingerprint di dataset e benchmark dove i protocolli ufficiali lo richiedono. Il valutatore delle composizioni classiche controlla il fingerprint di congelamento 57ff2b7c725a4ef802817581fac9430f8c6416a6de54dafcb1deb7b471716b60; il benchmark LLM controlla prompt, parser, digest del modello e identità del dataset prima di accettare un'esecuzione. Questi controlli rendono l'identità dell'esecuzione più precisa di una sola invocazione da riga di comando.

Gli artefatti di analisi finale aggregano valutazioni completate in tabelle destinate al report. Sono utili per ispezionare i numeri pubblicati, ma non sostituiscono i file dei checkpoint congelati o i manifesti dei benchmark quando si riproducono le valutazioni sottostanti.

F.5 Ambiente e limiti di un clone fresco

Il repository fissa seed, configurazioni ufficiali, manifesti degli split, hash dei checkpoint e fingerprint dei benchmark. Non garantisce un riaddestramento byte-identico in tutti gli ambienti. Kernel GPU, versioni delle librerie, comportamento mixed precision, scheduling dei data loader e dettagli implementativi di PyTorch/PyTorch Geometric possono influenzare le traiettorie esatte di ottimizzazione. Un clone fresco può inoltre richiedere artefatti grandi forniti esternamente o rigenerati, a seconda di quali dataset generati e checkpoint siano inclusi nella distribuzione pubblica. La dichiarazione di riproducibilità è quindi più forte per l'ispezione degli artefatti e la valutazione dei checkpoint congelati, e più debole per l'equivalenza byte-per-byte del riaddestramento.